Documento de Diseño

Arquitectura, modelo de datos, UML, requisitos no funcionales y Docker

En este Documento se describe cómo está construido el módulo administrativo de FarmaClick: componentes, comunicación entre servicios, datos, comportamiento y despliegue

Proyecto	FarmaClick - Sistema de gestión para farmacias - Módulo administrativo
Cliente / contraparte	MediFarm
Equipo	D-Software: Alejandro Fuentes Navarrete, Víctor Rubilar Inostroza, Robert González Muñoz
Asignatura	PTY4614 - Capstone
Metodología	Scrum (sprints de 2 semanas)
Código del documento	DOC-07
Versión	1.1
Fecha	16 de octubre de 2026
Repositorio	https://github.com/Alefuentes982/FarmaClick

Control de versiones

Versión	Fecha	Autor	Cambios
1.0	21 de agosto de 2026	Alejandro Fuentes (PO)	Versión inicial (fase de inicio): arquitectura, stack y modelo de datos.
1.1	16 de octubre de 2026	Alejandro Fuentes (PO)	Actualizado al Incremento 4: despliegue en Oracle Cloud con Docker, Supabase dev y prod, migraciones y diagramas UML del código consolidado.

Contenido

1. Introducción 5
2. Arquitectura 6
2.1 Diagrama de arquitectura 6
2.2 Componentes 7
2.3 Comunicación entre servicios 8
3. Vista de despliegue 9
4. Modelo de datos 10
4.1 Modelo entidad-relación 10
4.2 Tablas 16
4.3 Lógica en la base de datos 17
4.4 Seguridad del modelo 18
5. Diagramas UML 19
5.1 Casos de uso 19
5.2 Diagrama de clases 24
5.3 Diagramas de secuencia 26
5.4 Diagramas de estados 30
6. API REST 31
7. Requisitos no funcionales 32
8. Docker y ejecución 33
8.1 Dockerfiles 33
8.2 docker-compose 33
8.3 Variables de entorno 33
8.4 Cómo levantar el sistema 34

9. Decisiones de arquitectura.....34

Anexo A. Diccionario de datos.....34

1. Introducción

Este documento describe el diseño del **módulo administrativo** de FarmaClick tal como está implementado en el repositorio. Sirve como referencia para el equipo y la contraparte. El diseño de la **tienda en línea** se incorporará a este documento a medida que se construya: usa la misma arquitectura, un módulo *tienda* en la API con endpoints públicos de solo lectura, la misma base de datos (*vw_catalogo_web, cliente, pasarela, transaccion_pago*) y las reglas existentes de pedidos y reserva FEFO.

Elemento	Detalle
Frontend	React 19 + Vite 8 + React Router 7 + Bootstrap 5.3, servido por Nginx
Backend	Node.js 22 + Express 5, zod, node-postgres, pdfkit, exceljs, nodemailer, express-rate-limit, @supabase/supabase-js
Base de datos	PostgreSQL 16 en Supabase (39 tablas, 2 vistas, funciones y triggers, pg_cron)
Autenticación	Supabase Auth (JWT); permisos propios por módulo
Despliegue	Docker - GitHub Actions - GHCR - Oracle Cloud (2 VM E2.1.Micro + balanceador HTTPS) - Supabase
Repositorio	https://github.com/Alefuentes982/FarmaClick

2. Arquitectura

2.1 Diagrama de arquitectura

FarmaClick sigue una **arquitectura en capas** (cliente-presentación-aplicación-datos) con una **API REST modular**: cada módulo de negocio se registra en el backend y en el frontend con su clave de permiso, de modo que sumar un módulo (por ejemplo, la tienda en línea) no obliga a modificar los existentes.

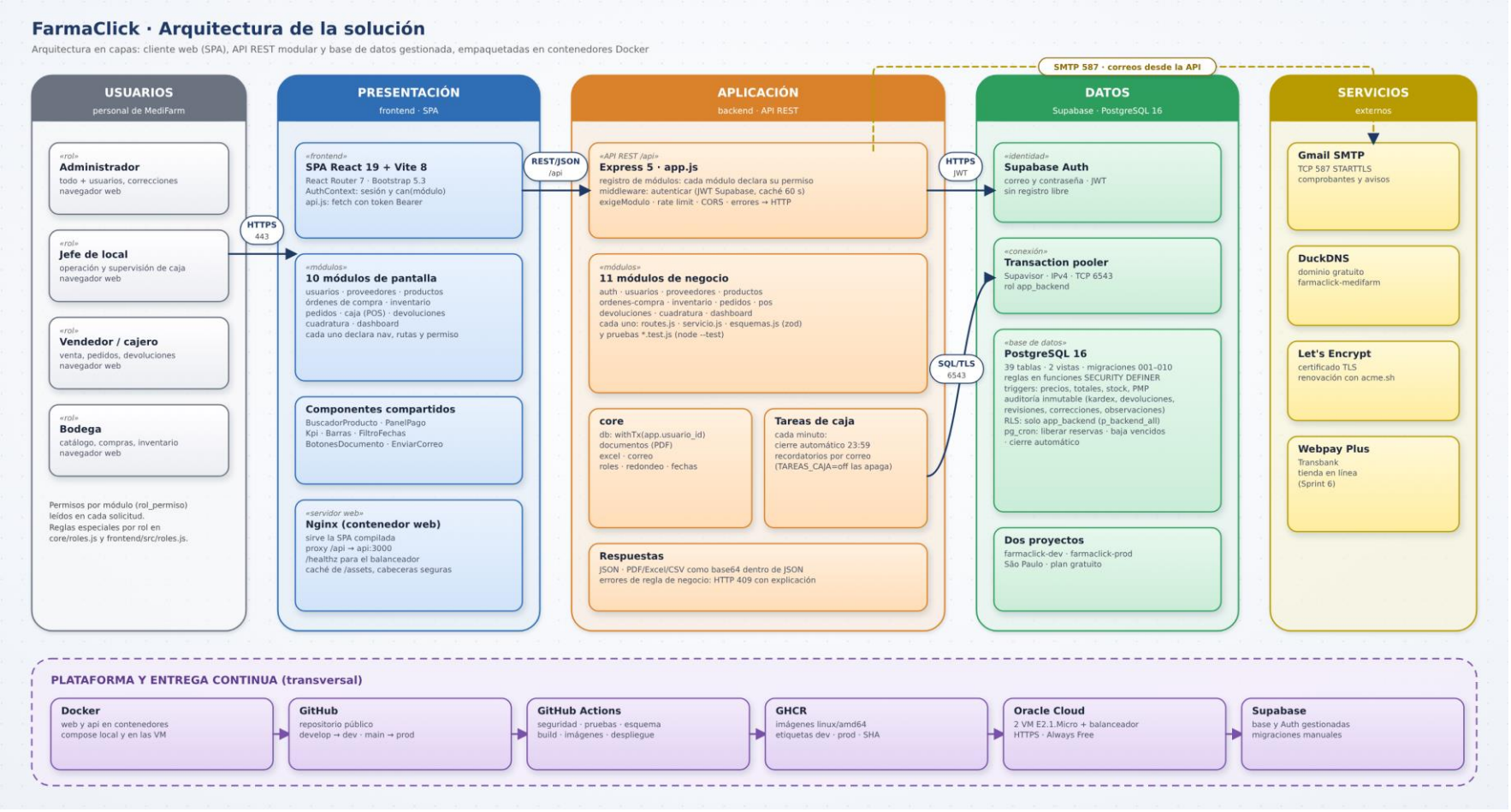


Figura 1. Arquitectura de la solución.

2.2 Componentes

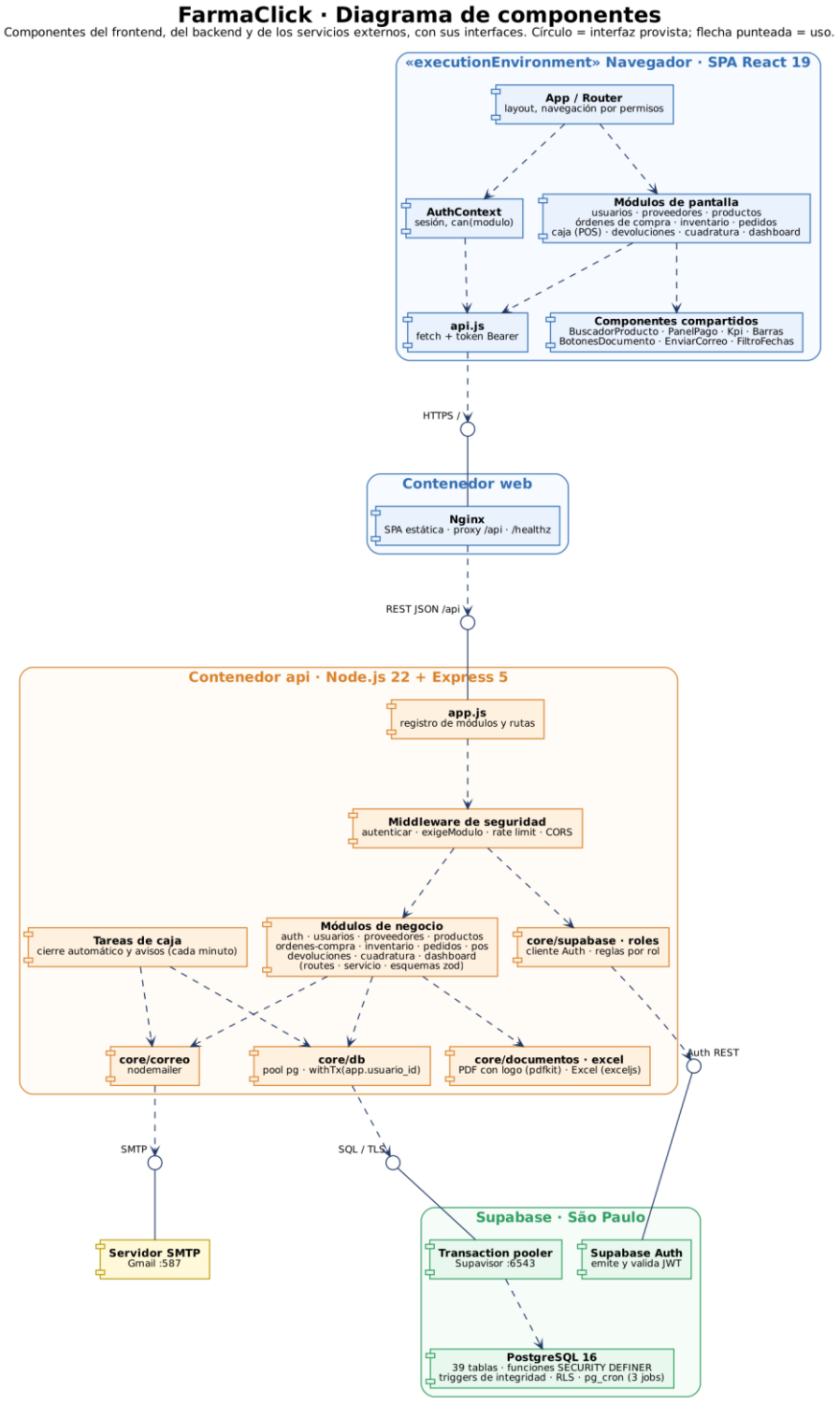


Figura 2. Diagrama de componentes (UML).

Componente	Responsabilidad	Tecnología
SPA (App, AuthContext, api.js)	Navegación según permisos, sesión, llamadas a la API con token Bearer	React 19, React Router 7
Módulos de pantalla (10)	Pantallas por módulo; cada uno declara nav, rutas y permiso	React, Bootstrap 5.3
Nginx	Sirve la SPA, reenvía /api a la API, /healthz, cabeceras de seguridad	nginx 1.27-alpine
app.js y middleware	Registro de módulos, autenticación (JWT), autorización por módulo, límite de intentos, CORS, manejo de errores	Express 5
Módulos de negocio (11)	routes.js (HTTP), servicio.js (reglas), esquemas.js (validación zod), pruebas	Node.js 22
core/db	Pool de conexiones y withTx: transacción con app.usuario_id para auditoría	node-postgres
core/documentos y excel	PDF con logo (OC, comprobantes, Cierre Z) y Excel	pdfkit, exceljs
core/correo y tareas de caja	Comprobantes por correo, cierre automático y recordatorios	nodemailer
Supabase Auth	Emisión y validación de JWT	Supabase
PostgreSQL	Datos, reglas (funciones), integridad (triggers), RLS, jobs	PostgreSQL 16, pg_cron

2.3 Comunicación entre servicios

Origen	Destino	Protocolo - puerto	Autenticación	Qué viaja
Navegador	Balanceador OCI (prod)	HTTPS - 443	TLS (Let's Encrypt)	SPA y llamadas /api
Navegador	VM dev	HTTP - 80	-	Entorno de pruebas del equipo
Balanceador	Nginx (VM prod)	HTTP - 80	Red privada de la VCN (NSG)	Tráfico descifrado; health check /healthz
Nginx	API	HTTP - 3000	Red interna de Docker	Rutas /api/*
SPA	API	REST/JSON	Bearer JWT	Datos; PDF/Excel/CSV como base64 en JSON
API	Supabase Auth	HTTPS - 443	anon/service role key	Login, validación de token, creación de cuentas
API	Transaction pooler	PostgreSQL/TLS - 6543	Usuario app_backend.<ref>	Consultas SQL (sin sentencias con nombre)
Pooler	PostgreSQL	PostgreSQL - 5432	Interna de Supabase	Conexiones a la base
API	Gmail SMTP	SMTP STARTTLS - 587	Contraseña de aplicación	Comprobantes y avisos de caja
GitHub Actions	VM dev/prod	SSH - 22	Llave farmaclick_deploy	Copia del compose y docker compose up
VM	GHCR	HTTPS - 443	Paquete accesible	docker pull de imágenes

3. Vista de despliegue

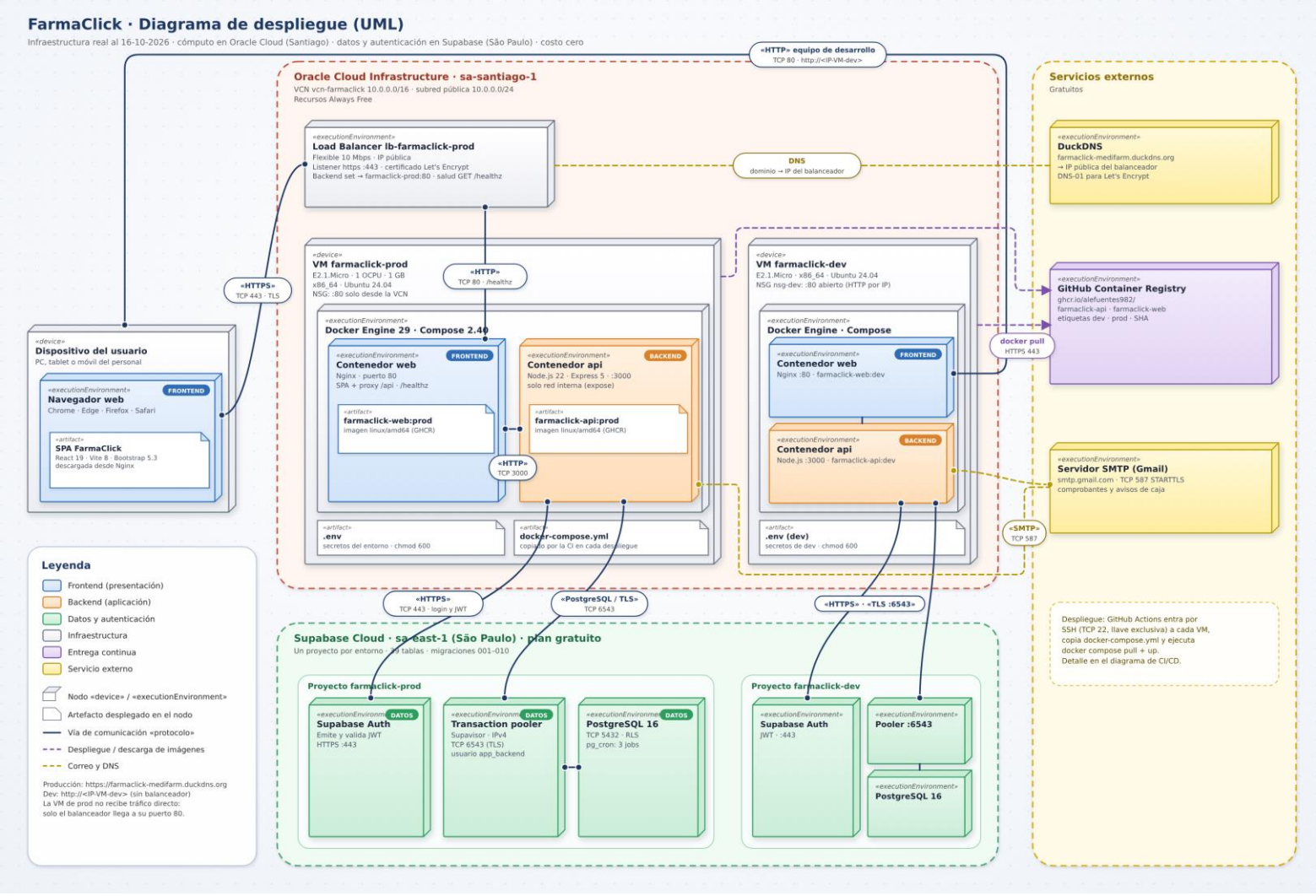


Figura 3. Diagrama de despliegue (UML) con la infraestructura real.

Nodo	Tipo - tamaño	Contenido	Red
lb-farmaclick-prod	Load Balancer flexible 10 Mbps (Always Free)	Listener https:443 con certificado Let's Encrypt; backend set > farmaclick-prod:80; salud /healthz	Subred pública; IP pública con dominio DuckDNS
farmaclick-prod	VM.Standard.E2.1.Micro - 1 OCPU - 1 GB - x86_64 - Ubuntu 24.04	Docker: contenedores web (:80) y api (:3000); .env (600)	nsg-prod: :80 solo desde 10.0.0.0/16; SSH 22
farmaclick-dev	VM.Standard.E2.1.Micro - igual	Docker: web y api con etiqueta dev	nsg-dev: :80 abierto; SSH 22
Supabase farmaclick-prod / -dev	Proyectos gratuitos - São Paulo	Auth, pooler y PostgreSQL 16 con 3 jobs de pg_cron	Pooler IPv4 :6543
GHCR	GitHub Container Registry	farmaclick-api y farmaclick-web (dev, prod, SHA)	HTTPS

4. Modelo de datos

4.1 Modelo entidad-relación

El modelo es relacional (PostgreSQL) y está normalizado en tercera forma normal. Se organiza en siete áreas. La vista general muestra solo claves; los diagramas por área muestran todas las columnas con su tipo. Las tablas de catálogo (estados, medios de pago, motivos) permiten agregar valores sin cambiar el código.

El modelo se puede revisar en detalle en la carpeta "Diagramas/Modelo" donde se encuentran un archivo html interactivo y otro de imagen con el ERD.

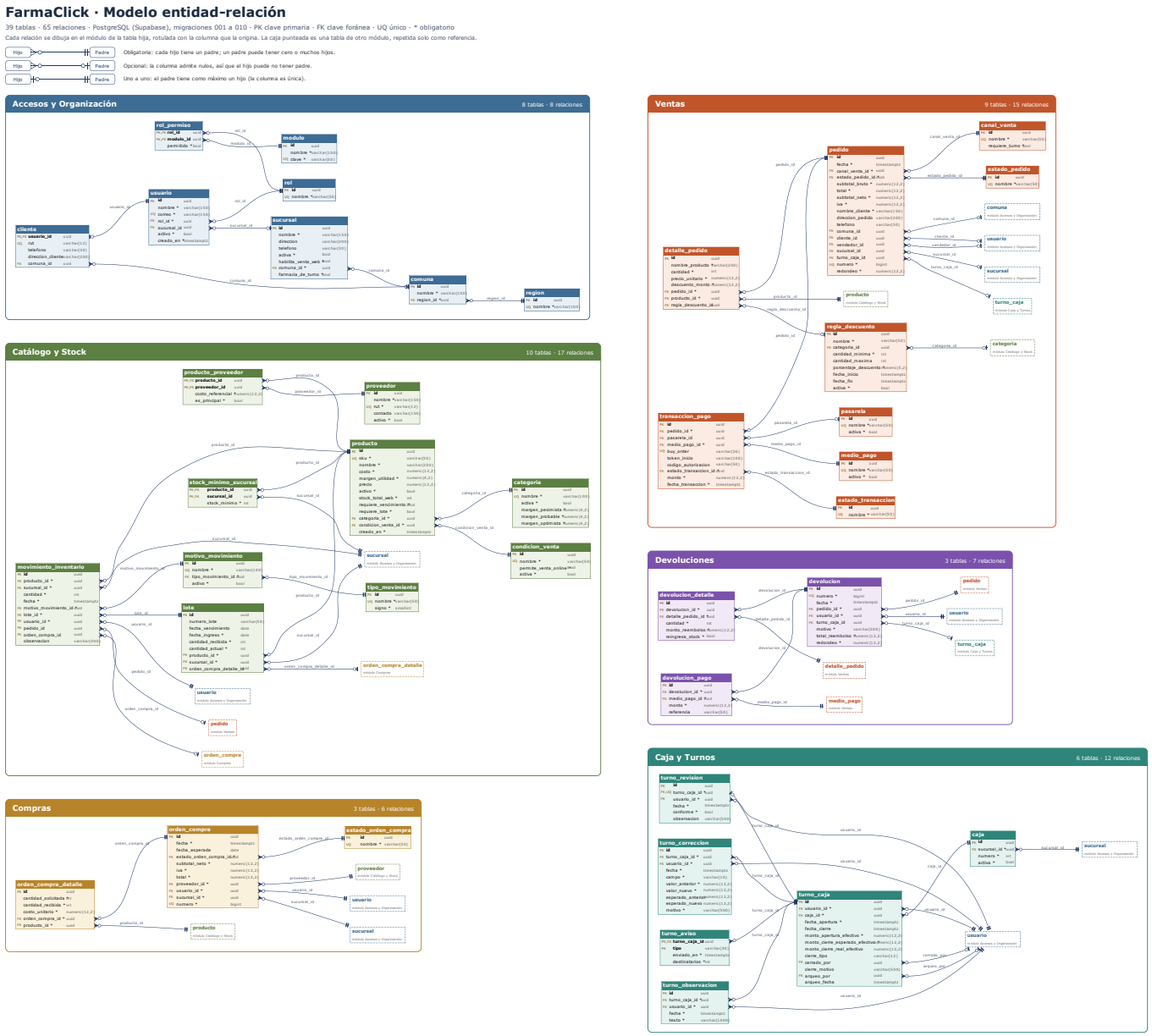
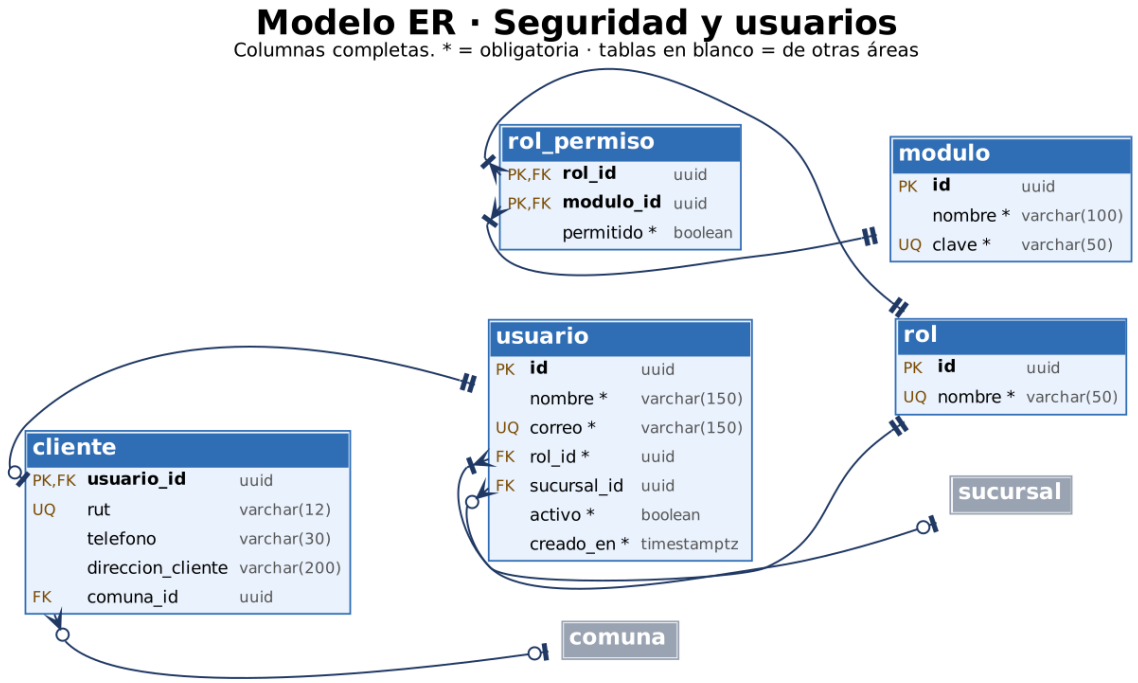
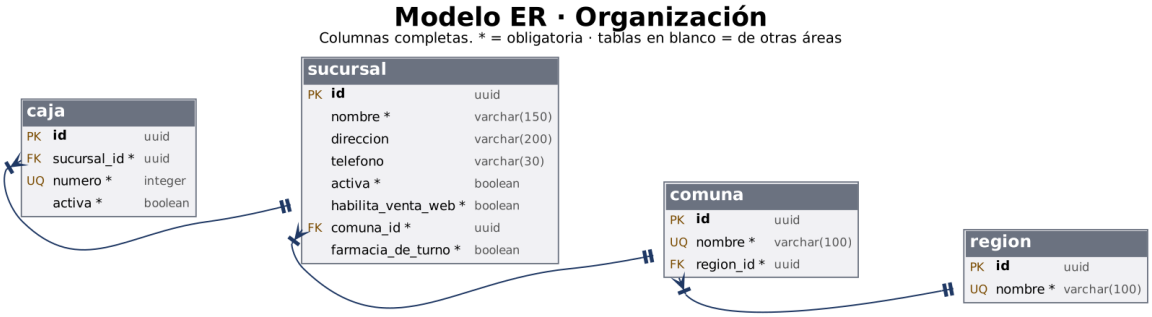


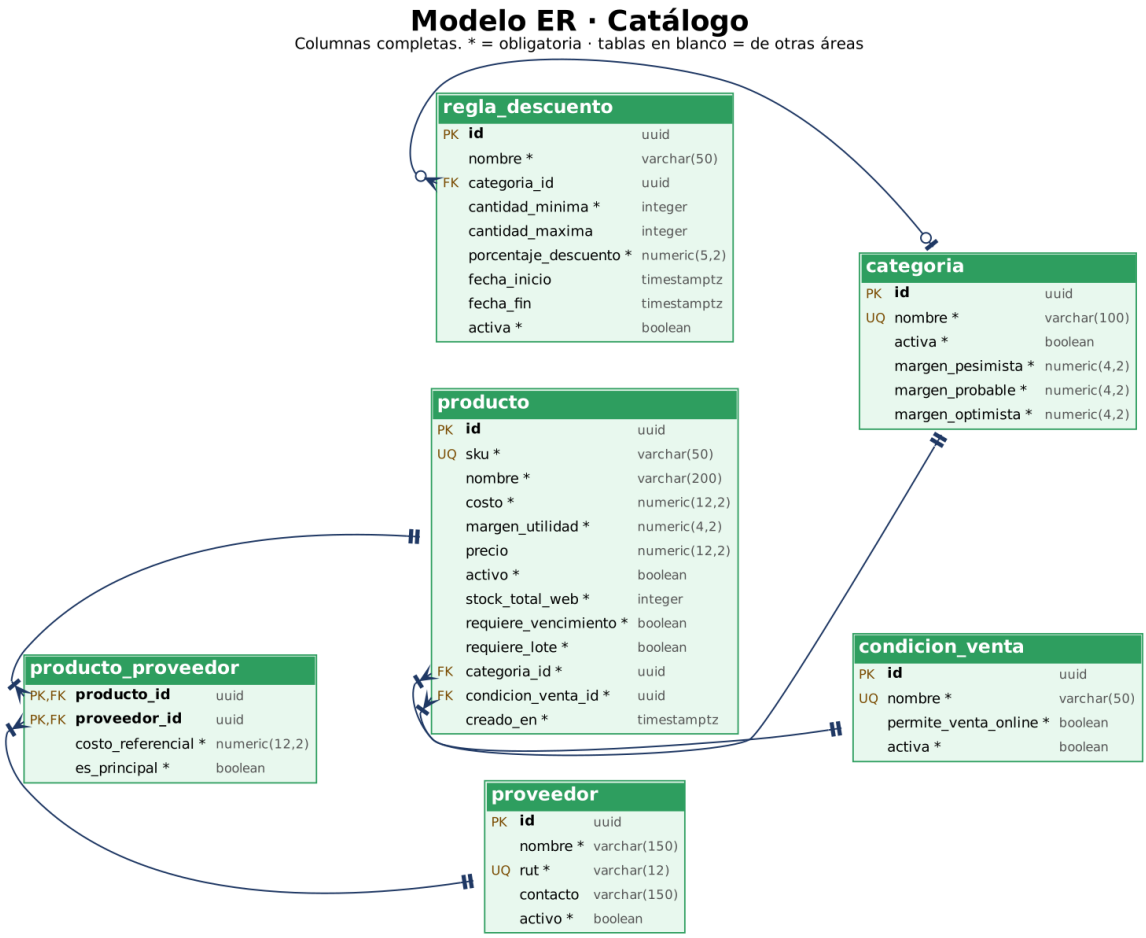
Figura 4. Modelo entidad-relación, vista general (39 tablas).



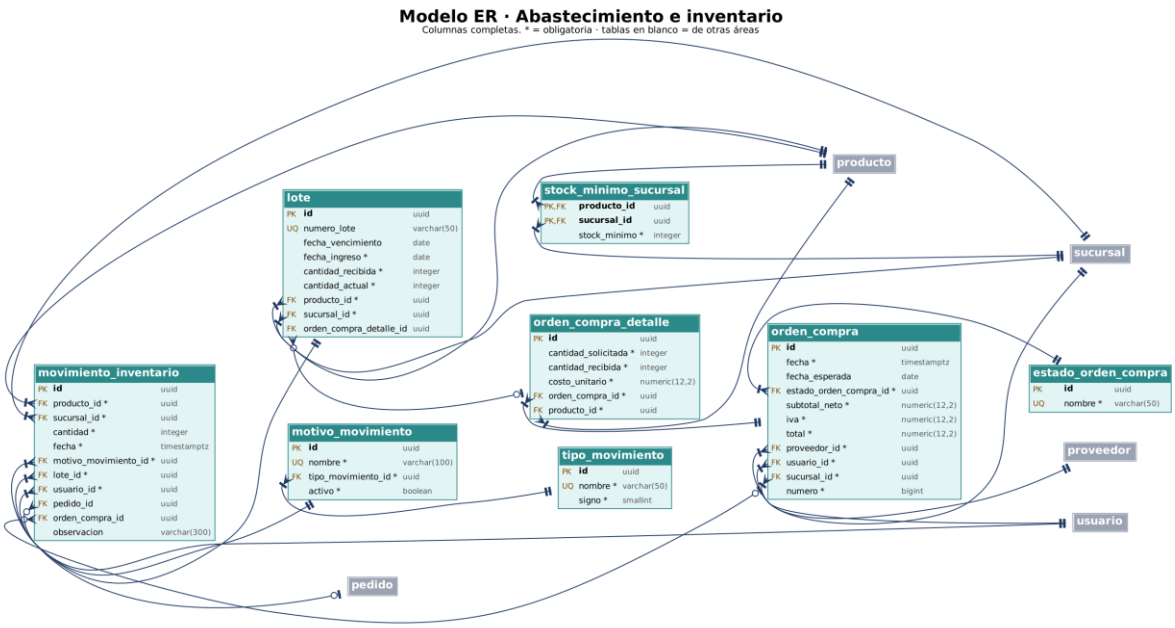
Modelo ER - Seguridad y usuarios



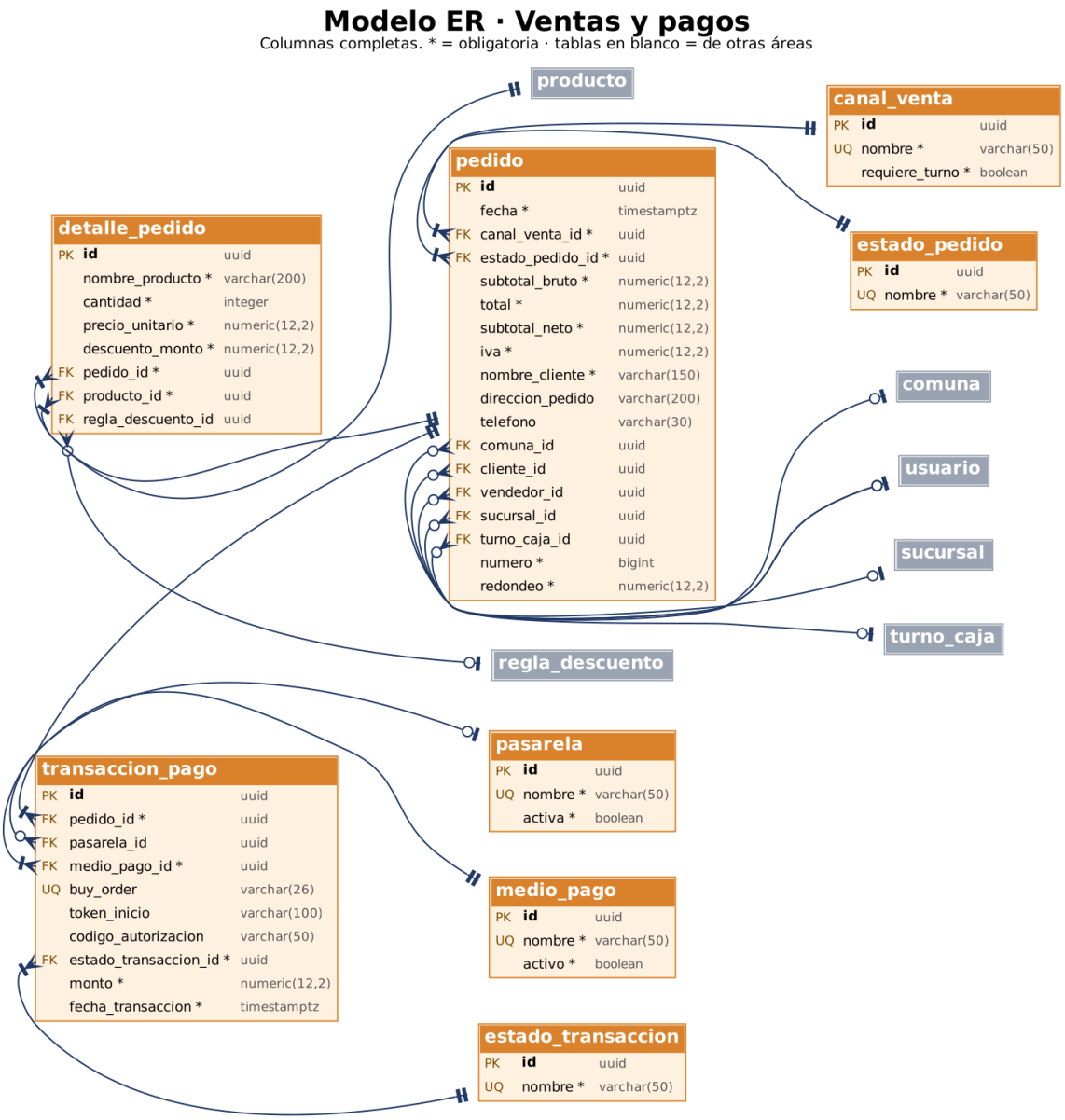
Modelo ER - Organizacion



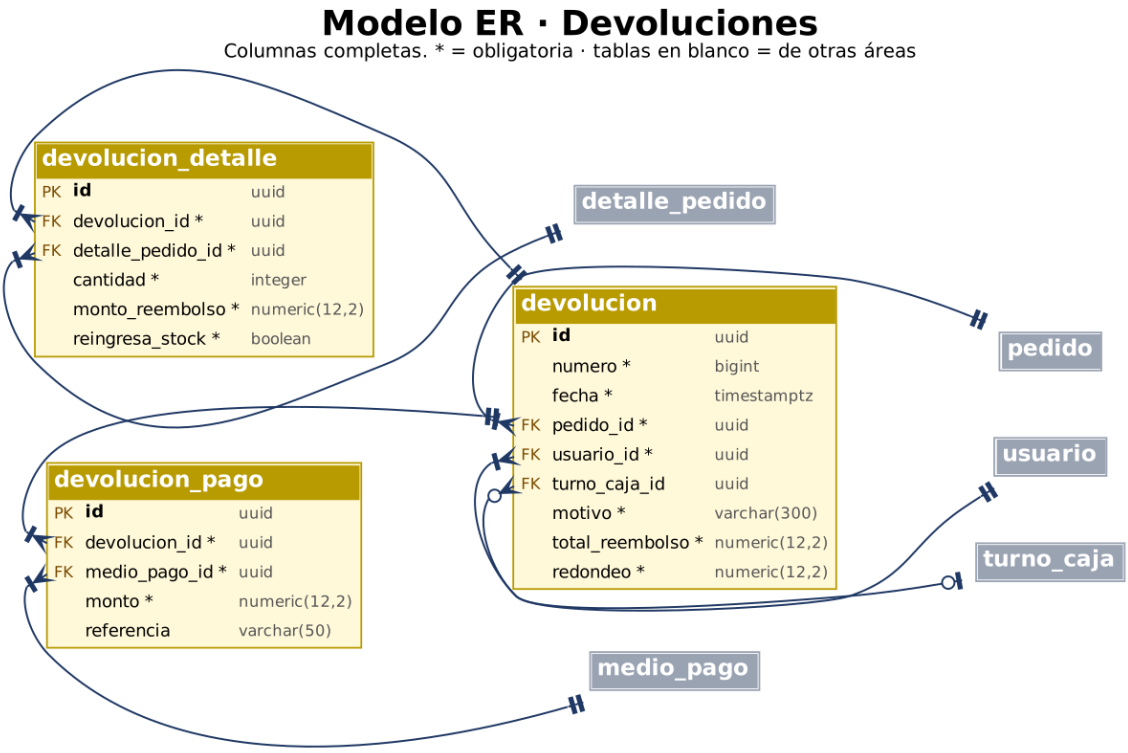
Modelo ER - Catalogo



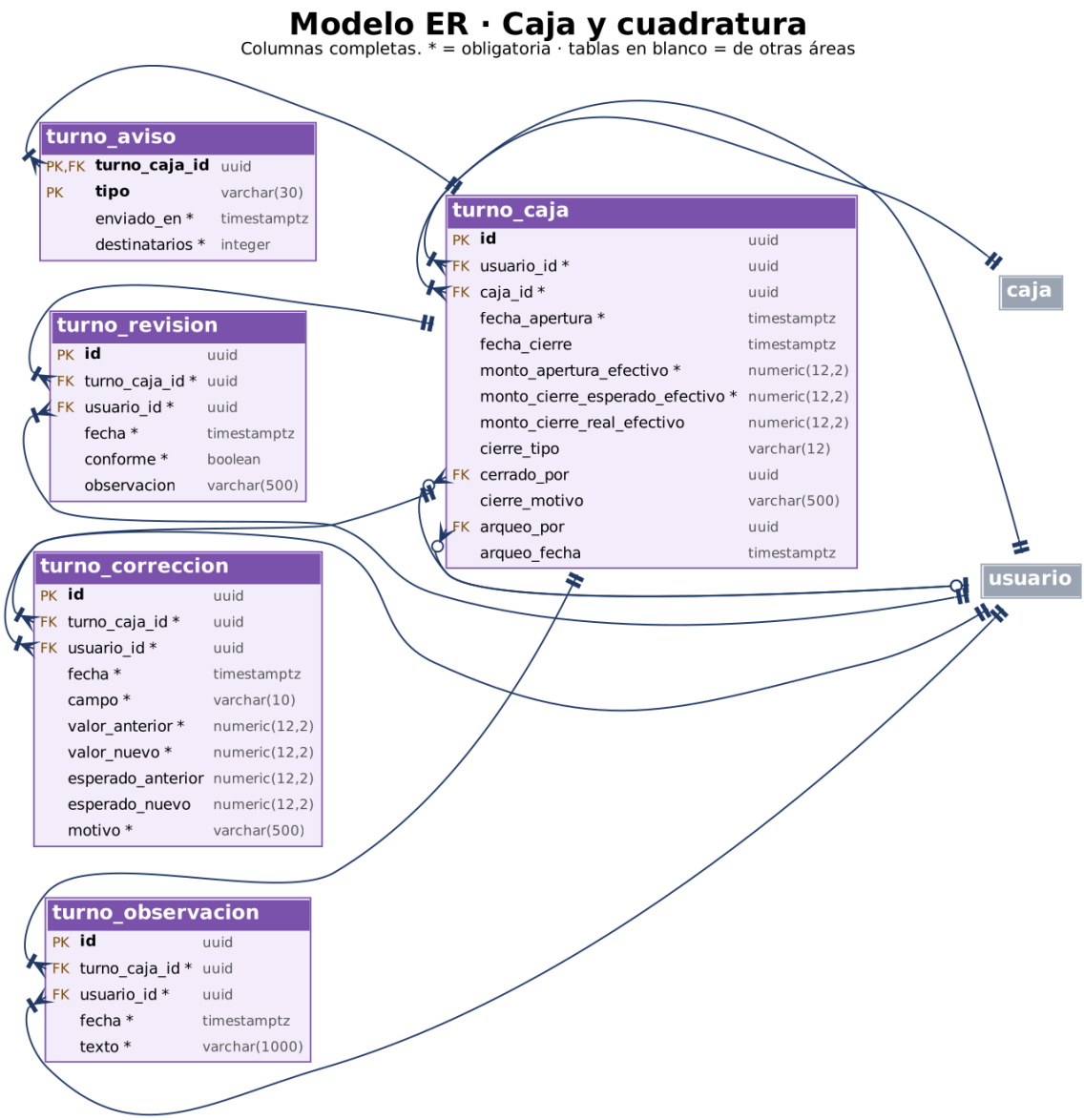
Modelo ER - Abastecimiento e inventario



Modelo ER - Ventas y pagos



Modelo ER - Devoluciones



Modelo ER - Caja y cuadratura

4.2 Tablas

Tabla	Descripción	Columnas
caja	Cajas físicas de cada sucursal.	4
canal_venta	Presencial (exige turno) u Online.	3
categoria	Categorías con márgenes pesimista, probable y optimista (PERT).	6
cliente	Datos del cliente final (para la tienda en línea).	5
comuna	Comunas de Chile.	3
condicion_venta	Condición de venta (venta directa, receta, etc.) y si permite venta en línea.	4
detalle_pedido	Líneas del pedido con precio y descuento fijados por la base.	8
devolucion	Devolución inmutable de un pedido entregado con total y redondeo.	9
devolucion_detalle	Líneas devueltas, reembolso y si reingresa stock.	6
devolucion_pago	Reembolso por medio de pago.	5

Tabla	Descripción	Columnas
estado_orden_compra	Borrador, Pendiente, Recibida parcial, Recibida, Cancelada.	2
estado_pedido	Pago pendiente, Pagado, En preparación, Listo para retiro, En despacho, Entregado, Cancelado, Expirado.	2
estado_transaccion	Estados de un pago (Aprobado, Anulado, Abortado...).	2
lote	Lotes por producto y sucursal con vencimiento y cantidad actual.	9
medio_pago	Efectivo, Débito, Crédito, Transferencia, Webpay Plus.	3
modulo	Módulos con clave de permiso (dashboard, pos, inventario, proveedores, pedidos, cuadratura, usuarios, devoluciones, supervision_caja).	3
motivo_movimiento	Motivos (recepción, venta, ajuste, merma, vencido, devolución...).	4
movimiento_inventario	Kardex inmutable: cada entrada y salida con motivo, lote y usuario.	11
orden_compra	Orden de compra con número correlativo, estado y totales.	11
orden_compra_detalle	Líneas de la OC: solicitado, recibido y costo.	6
pasarela	Pasarelas de pago (Webpay Plus, tienda en línea).	3
pedido	Venta (presencial u online) con número, estado, totales, redondeo y turno de caja.	18
producto	Catálogo: SKU, costo, margen, precio calculado, control de lote y vencimiento.	13
producto_proveedor	Proveedores de cada producto, costo referencial y proveedor principal.	4
proveedor	Proveedores con RUT validado.	5
region	Regiones de Chile.	2
regla_descuento	Descuentos por cantidad, opcionalmente por categoría, con vigencia.	9
rol	Roles: Administrador, Vendedor, Bodega, Cliente, Sistema y Jefe de local.	2
rol_permiso	Matriz de permisos rol x módulo.	3
stock_minimo_sucursal	Stock mínimo por producto y sucursal.	3
sucursal	Locales de la farmacia; farmacia_de_turno = atiende 24 h.	8
tipo_movimiento	Entrada o salida (signo).	3
transaccion_pago	Pagos del pedido por medio y pasarela, con estado.	10
turno_aviso	Avisos de cierre enviados (evita repetirlos).	4
turno_caja	Turno de caja: apertura, cierre, esperado, contado, tipo de cierre y arqueo.	13
turno_correccion	Correcciones de montos con motivo (inmutable).	10
turno_observacion	Observaciones del turno (inmutables).	5
turno_revision	Revisión conforme u observada (inmutable).	6
usuario	Personal del sistema (mismo id que la cuenta de Supabase Auth), con rol y sucursal.	7

El **diccionario de datos completo** (cada columna con tipo, nulidad y clave) está en el anexo A de este documento.

4.3 Lógica en la base de datos

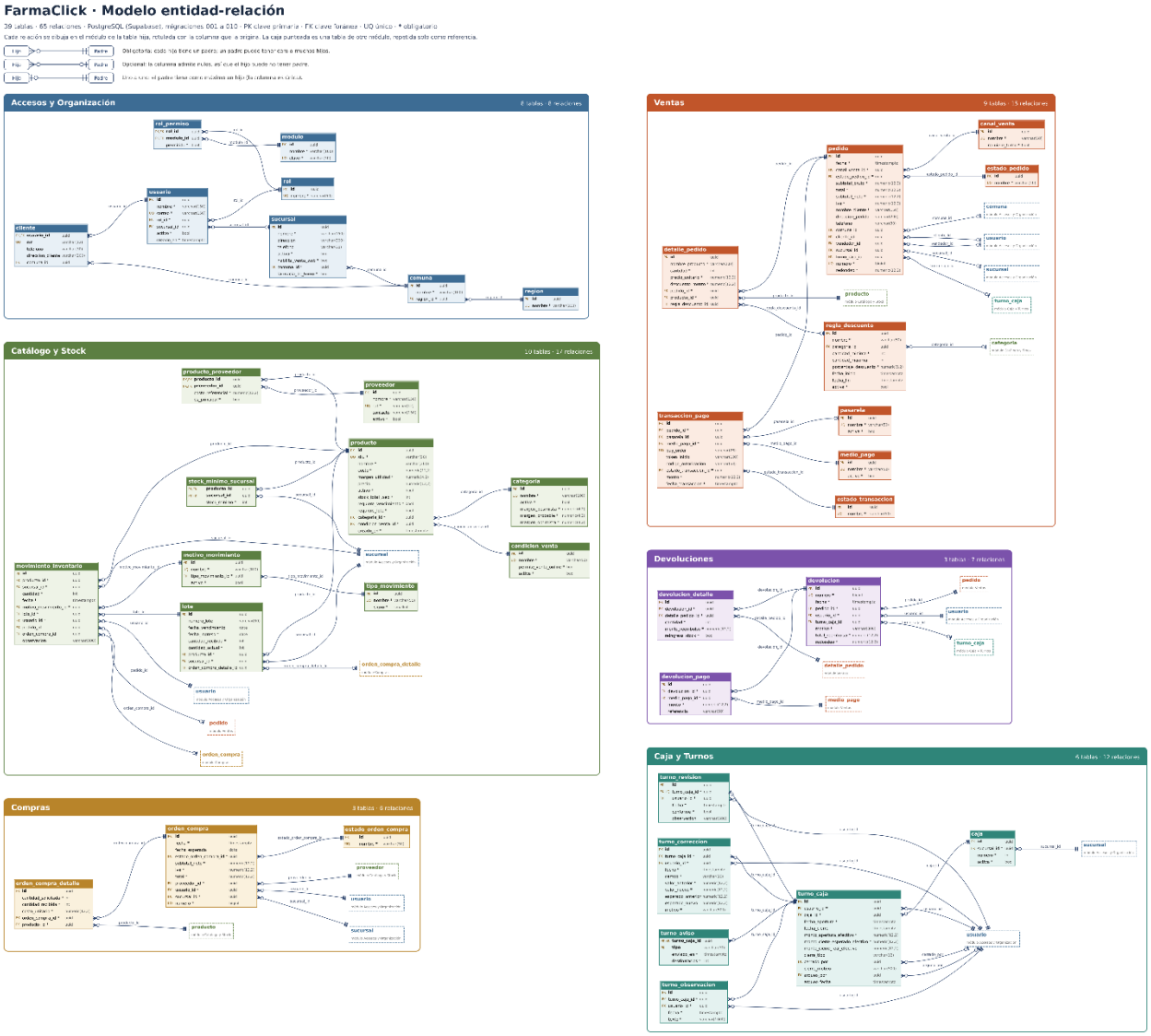
Tipo	Objeto	Qué hace
Función	fn_reservar_pedido / fn_liberar_reserva	Reserva stock por FEFO (movimientos "Venta") y lo devuelve ("Liberación de reserva").
Función	fn_liberar_reservas_vencidas	Expira pedidos sin pagar tras 20 minutos y libera su stock (job cada 5 min).
Función	fn_baja_lotes_vencidos	Da de baja los lotes vencidos (job diario 05:00 UTC).
Función	fn_cierre_esperado_efectivo	Calcula el efectivo esperado: apertura + ventas - devoluciones en efectivo.
Función	fn_cerrar_turno / fn_cerrar_turno_forzado	Cierre normal (solo quien abrió) y por fuerza mayor (Admin o Jefe de local, con motivo).
Función	fn_cierre_automatico_turnos	Cierra a las 23:59 (hora de Chile) los turnos abiertos salvo farmacias de turno (job 59 * * * * y API).
Función	fn_registrar_arqueo / fn_corregir_turno	Arqueo de turnos pendientes; corrección de montos solo por el Administrador.
Función	fn_es_administrador /	Reglas por rol usadas por otras funciones.

Tipo	Objeto	Qué hace
	fn_supervisa_caja	
Función	fn_programar_jobs	Crea o actualiza los 3 jobs de pg_cron.
Trigger	calcular_linea / recalcular_pedido / recalcular_oc	Fija precio y descuento de cada línea y recalcula totales (neto, IVA, total).
Trigger	trg_01_pmp / trg_02_stock	Recalcula el costo promedio ponderado y el stock en cada movimiento.
Trigger	prevent_kardex_upd / inmutable_*	Impiden modificar o borrar kardex, devoluciones, revisiones, correcciones y observaciones.
Trigger	val_sanidad / val_devolucion_detalle / val_turno_revision	Validan lote y vencimiento, cantidades devueltas y revisiones cruzadas.
Trigger	cierre_turno / val_pedido_turno	Solo cierra quien abrió (salvo autorización); una venta presencial exige turno.
Vista	vw_stock_disponible / vw_catalogo_web	Stock disponible por sucursal; catálogo para la tienda en línea (Sprint 5).

4.4 Seguridad del modelo

- **RLS** activado en todas las tablas con la política *p_backend_all* para el rol *app_backend*, el único que usa la API; se revocan los permisos de *anon* y *authenticated.*
- Las funciones son *SECURITY DEFINER* con *SET search_path = public*. Quien actúa llega en *app.usuario_id*, que fija *withTx* en cada transacción.
- Los errores de regla de negocio se lanzan con *RAISE EXCEPTION* y la API los devuelve como **HTTP 409** con la explicación.

- Migraciones **numeradas e idempotentes** (001-010) dentro de BEGIN/COMMIT; *instalacion_completa.sql* se regenera con *generar-instalacion.sh*.



5. Diagramas UML

5.1 Casos de uso

Actores: **Vendedor/cajero**, **Jefe de local** (hereda del Vendedor), **Administrador** (hereda del Jefe de local), **Bodega/encargado de compras**, y los actores de sistema **Supabase Auth**, **Servidor SMTP** y **Reloj** (pg_cron y tareas de la API).

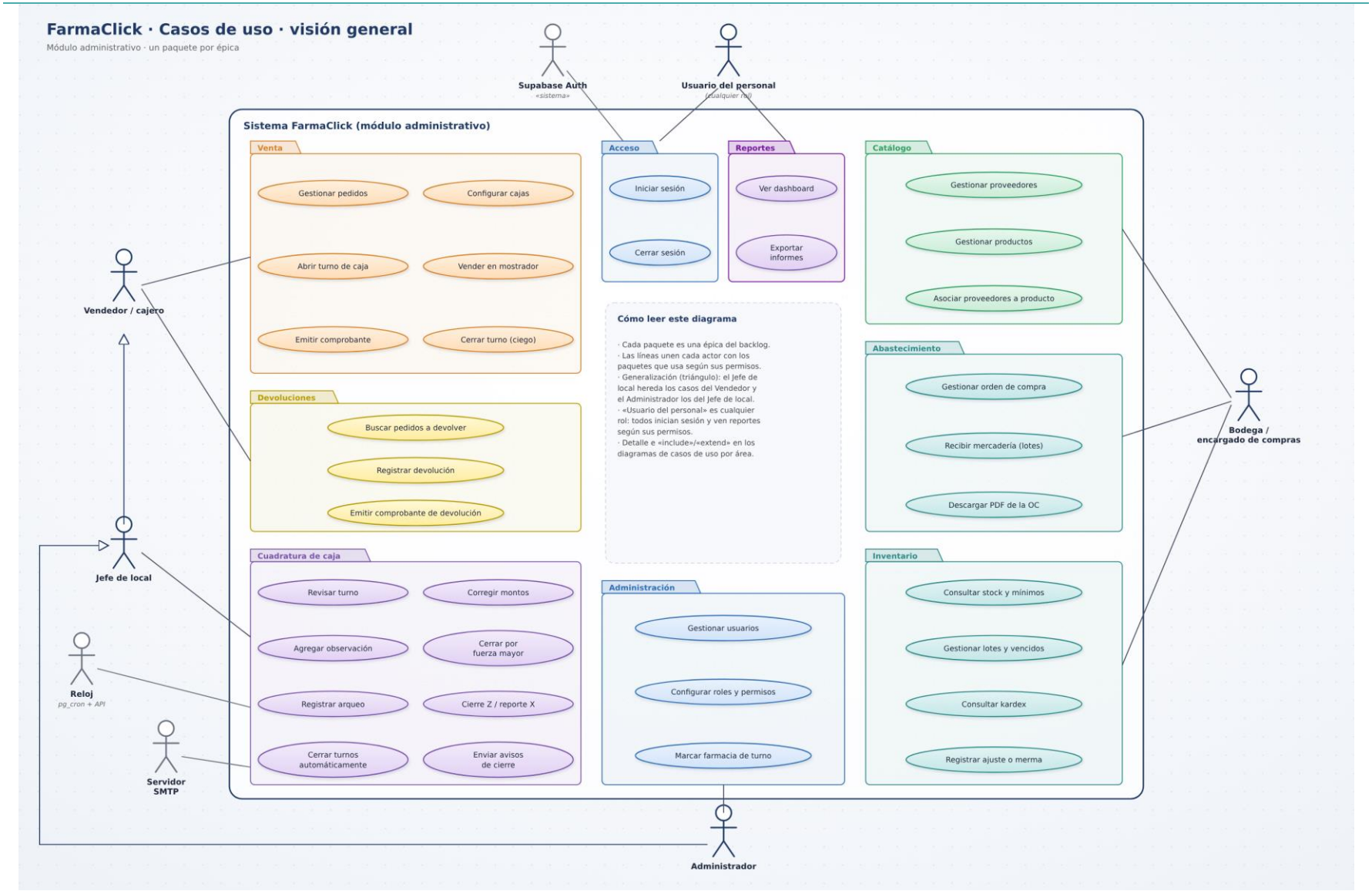


Figura 5. Casos de uso: visión general por paquetes (épicas).

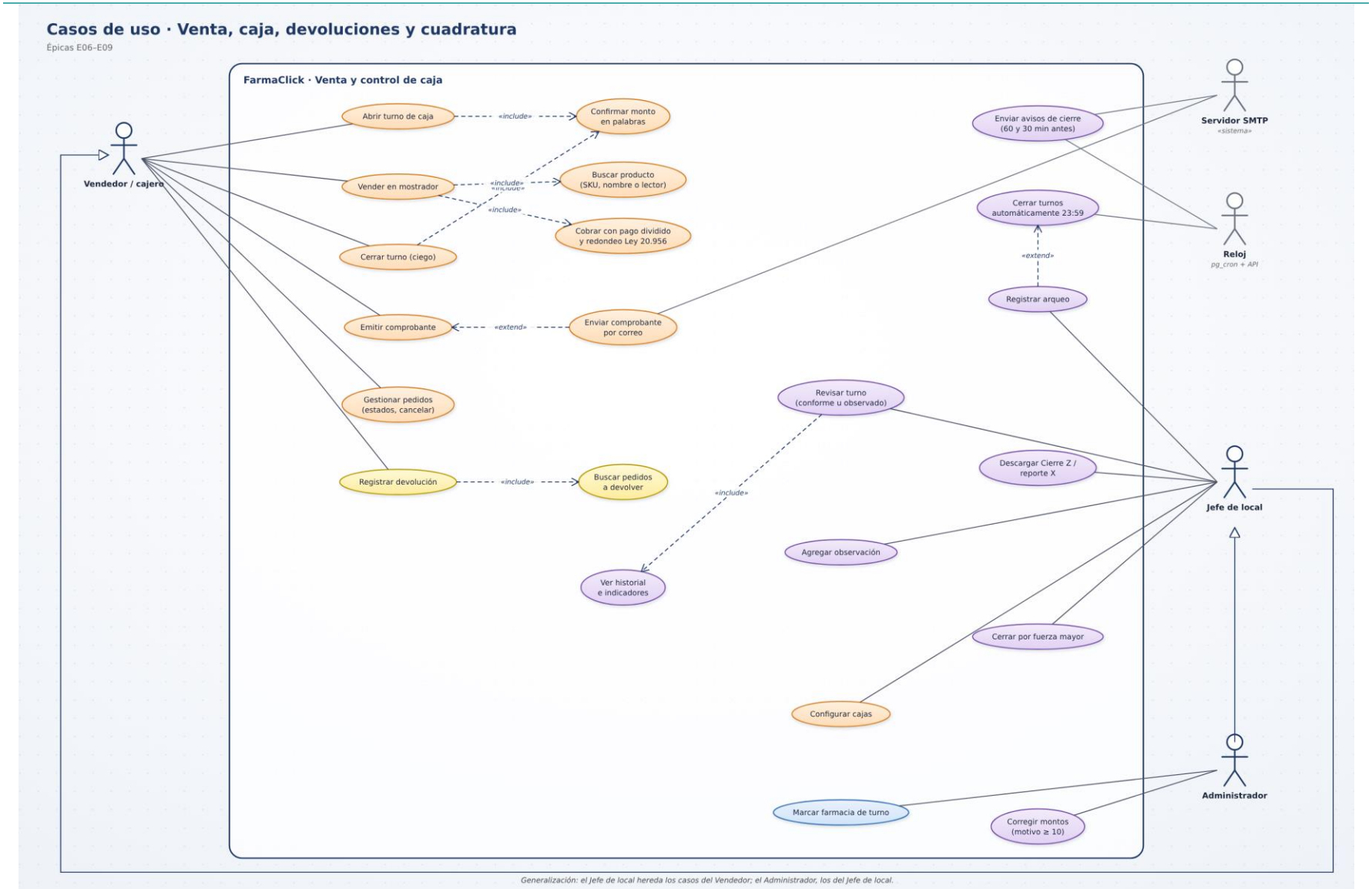


Figura 6. Casos de uso: venta, caja, devoluciones y cuadratura.

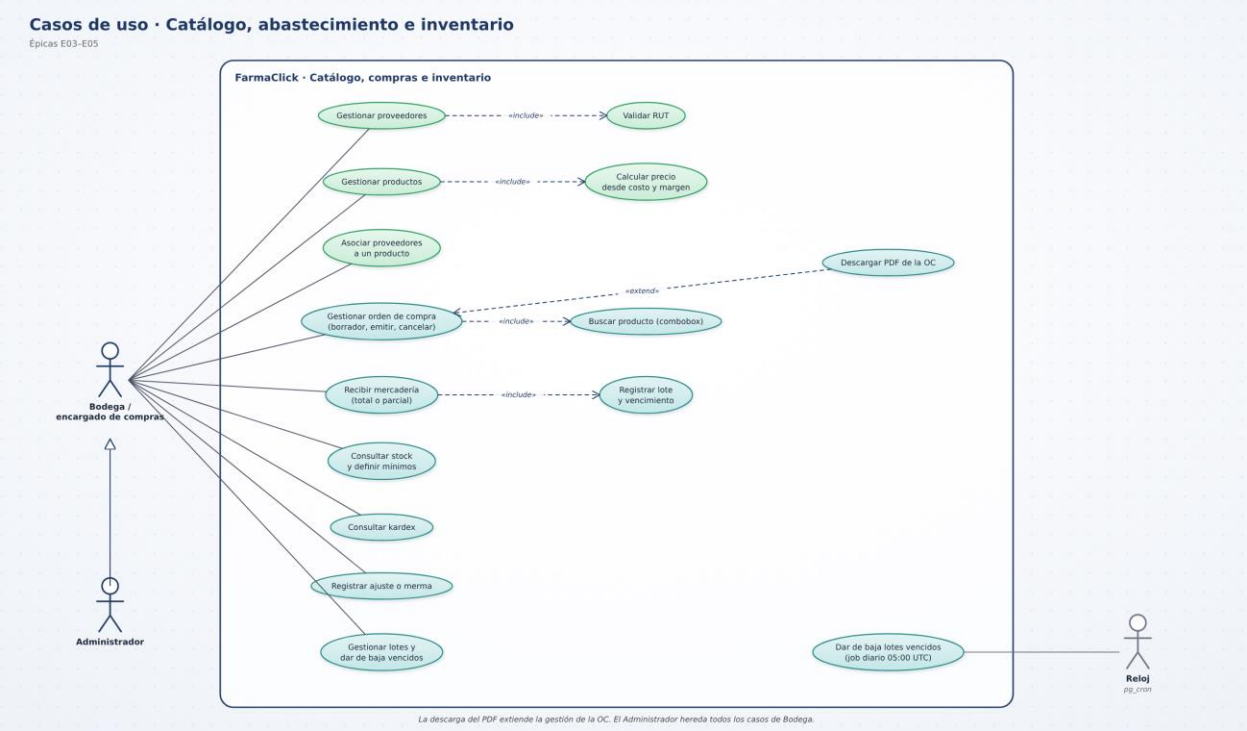


Figura 7. Casos de uso: catálogo, abastecimiento e inventario.

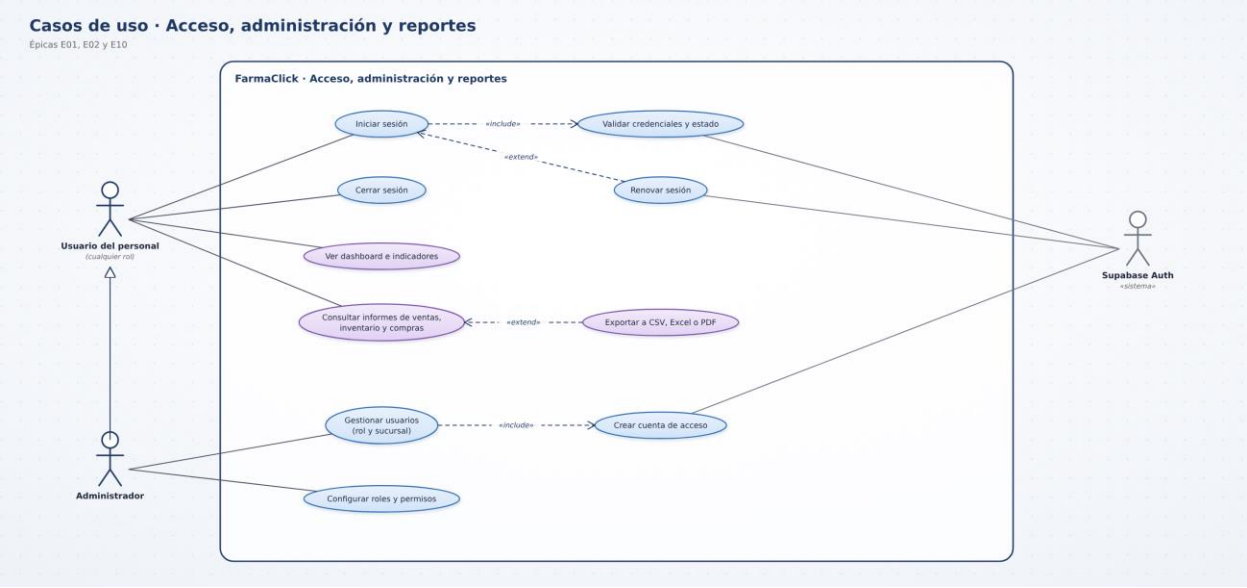


Figura 8. Casos de uso: acceso, administración y reportes.

Especificación de los casos de uso principales

CU-01 Vender en mostrador

Actor	Vendedor/cajero
Precondición	Tener un turno de caja abierto; productos con stock.
Flujo básico	• 1. El cajero busca productos por SKU, nombre o lector y los agrega al carrito. • 2. El sistema cotiza: precios, descuentos por cantidad, redondeo del efectivo (Ley 20.956) y vuelto. • 3. El cajero registra uno o más medios de pago y confirma. • 4. El sistema, en una sola transacción, crea el pedido, reserva stock por FEFO, registra los pagos y lo marca Entregado. • 5. El sistema muestra el comprobante (ver, descargar, imprimir o enviar por correo).
Flujos alternativos	• Sin turno abierto: 409 "Debe abrir un turno de caja". • Stock insuficiente o pagos que no cuadran: 409 con la explicación; no se guarda nada.
Postcondición	Pedido Entregado, kardex "Venta", pagos aprobados.

CU-02 Cerrar turno (ciego)

Actor	Vendedor/cajero
Precondición	Turno abierto por el mismo usuario.
Flujo básico	• 1. El cajero cuenta el efectivo y lo ingresa; el sistema lo muestra en palabras. • 2. El cajero confirma sin ver el esperado. • 3. El sistema calcula el esperado y la diferencia y cierra el turno.
Flujos alternativos	• Otro usuario intenta cerrar: rechazo (solo cierra quien abrió). • Si llega las 23:59 sin cierre: CU-04.
Postcondición	Turno Cerrado, pendiente de revisión.

CU-03 Revisar turno

Actor	Jefe de local / Administrador
Precondición	Turno cerrado y sin arqueo pendiente.
Flujo básico	• 1. El supervisor abre la ficha del turno: indicadores, desglose por medio de pago, historial. • 2. Marca Conforme u Observado (con observación obligatoria). • 3. El sistema registra la revisión (inmutable).
Flujos alternativos	• El cajero intenta revisar su propio turno: rechazo (revisión cruzada). • Montos mal digitados: el Administrador corrige antes de revisar (motivo >= 10 caracteres).
Postcondición	Turno Revisado.

CU-04 Cerrar turnos automáticamente

Actor	Reloj (pg_cron y API)
Precondición	Turnos abiertos en sucursales que no son farmacia de turno.
Flujo básico	• 1. 60 y 30 minutos antes, el sistema envía recordatorios por correo (una sola vez). • 2. A las 23:59 hora de Chile ejecuta fn_cierre_automatico_turnos(). • 3. Los turnos quedan con cierre_tipo "automatico" y pendientes de arqueo; se envía aviso.
Flujos alternativos	• Sin SMTP configurado: solo se hace el cierre. • Ejecutarlo más de una vez no tiene efecto.
Postcondición	Turnos pendientes de arqueo.

CU-05 Recibir mercadería

Actor	Bodega
Precondición	OC emitida (Pendiente o Recibida parcial).
Flujo básico	• 1. Bodega registra cantidades recibidas, lote y vencimiento por línea. • 2. El sistema crea lotes y movimientos "Recepción OC", actualiza stock y PMP. • 3. La OC queda Recibida o Recibida parcial.
Flujos alternativos	• Producto que exige vencimiento sin fecha: rechazo. • Recibir más de lo pedido: rechazo.
Postcondición	Stock y kardex actualizados.

CU-06 Registrar devolución

Actor	Vendedor/cajero
Precondición	Pedido Entregado dentro de 30 días.
Flujo básico	• 1. Busca el pedido con filtros. • 2. Elige productos y cantidades, motivo y si reingresa stock. • 3. El sistema cotiza el reembolso prorrateado por los medios originales. • 4. Confirma; se registra la devolución y el comprobante.
Flujos alternativos	• Reembolso en efectivo sin turno abierto: 409. • Cantidad mayor a la vendida: rechazo.
Postcondición	Devolución inmutable; efectivo descontado del cierre.

5.2 Diagrama de clases

Modelo de dominio: atributos principales y operaciones de negocio. Las operaciones se implementan en los servicios de la API (`servicio.js`) y en funciones de la base. Rombo negro = composición (la parte no existe sin el todo).

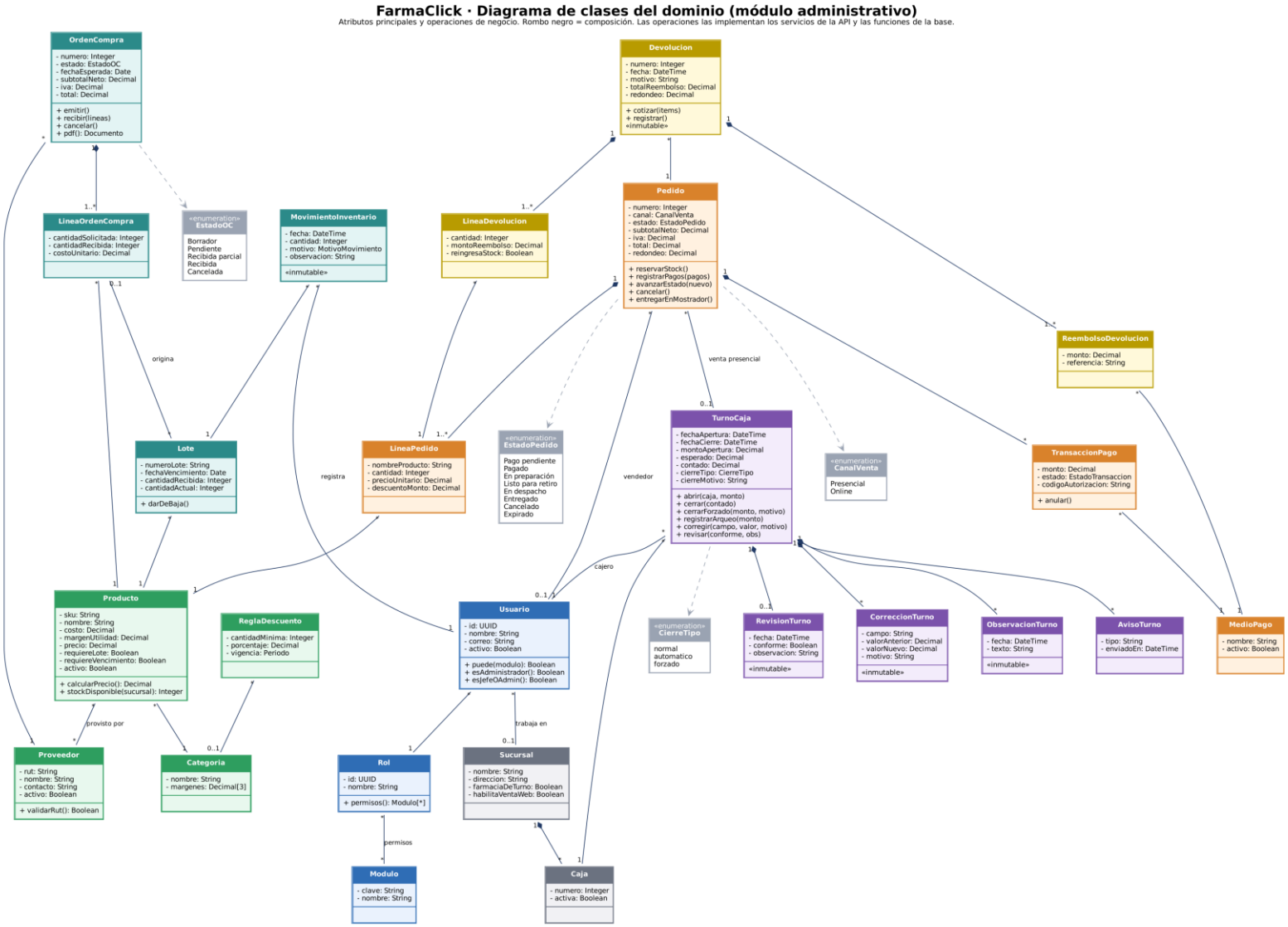


Figura 9. Diagrama de clases del dominio.

5.3 Diagramas de secuencia

La funcionalidad principal es la **venta en mostrador**. Se agregan el inicio de sesión con autorización por módulo, el cierre automático de caja y la recepción de una orden de compra.

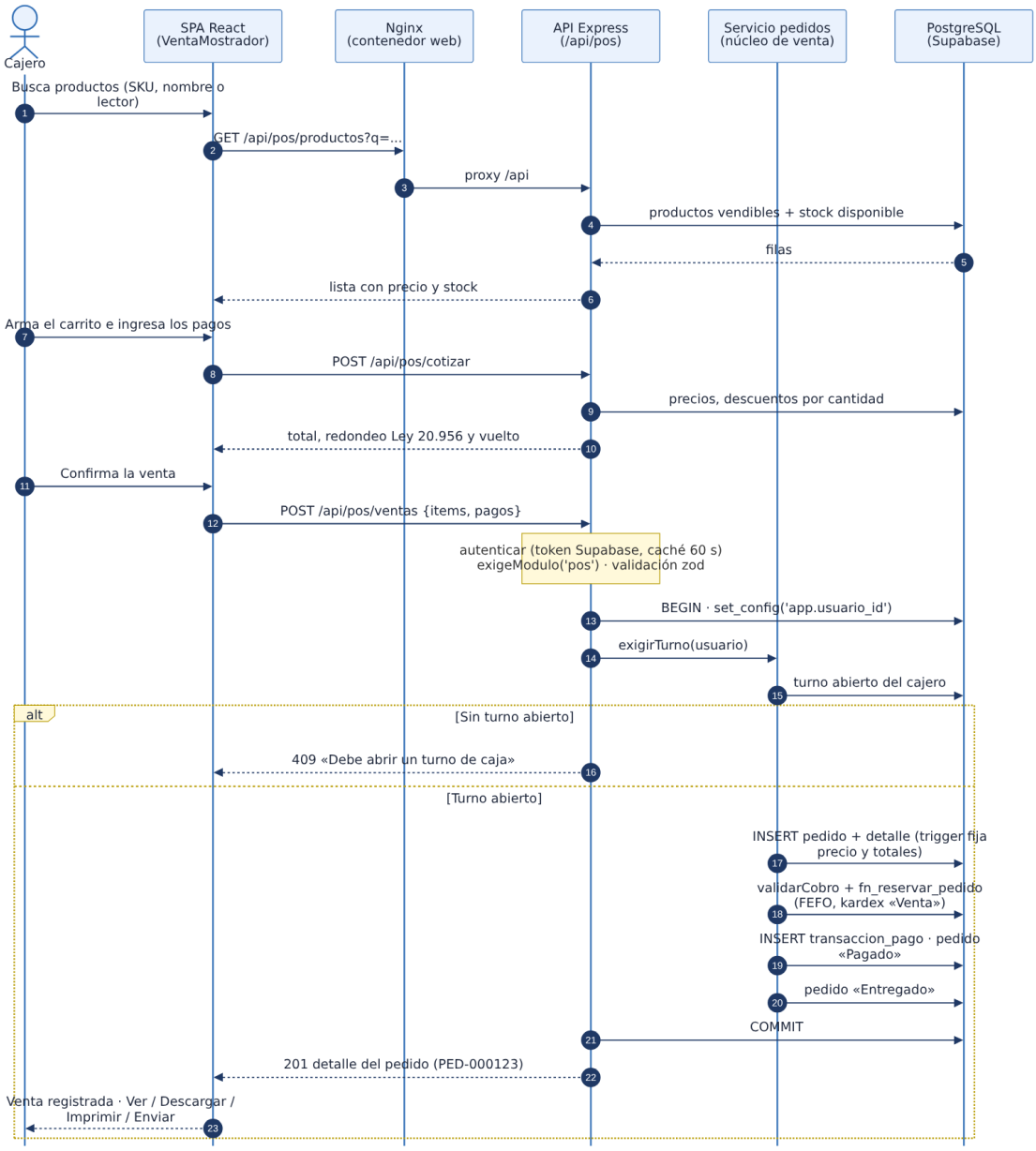


Figura 10. Secuencia: venta en mostrador (funcionalidad principal).

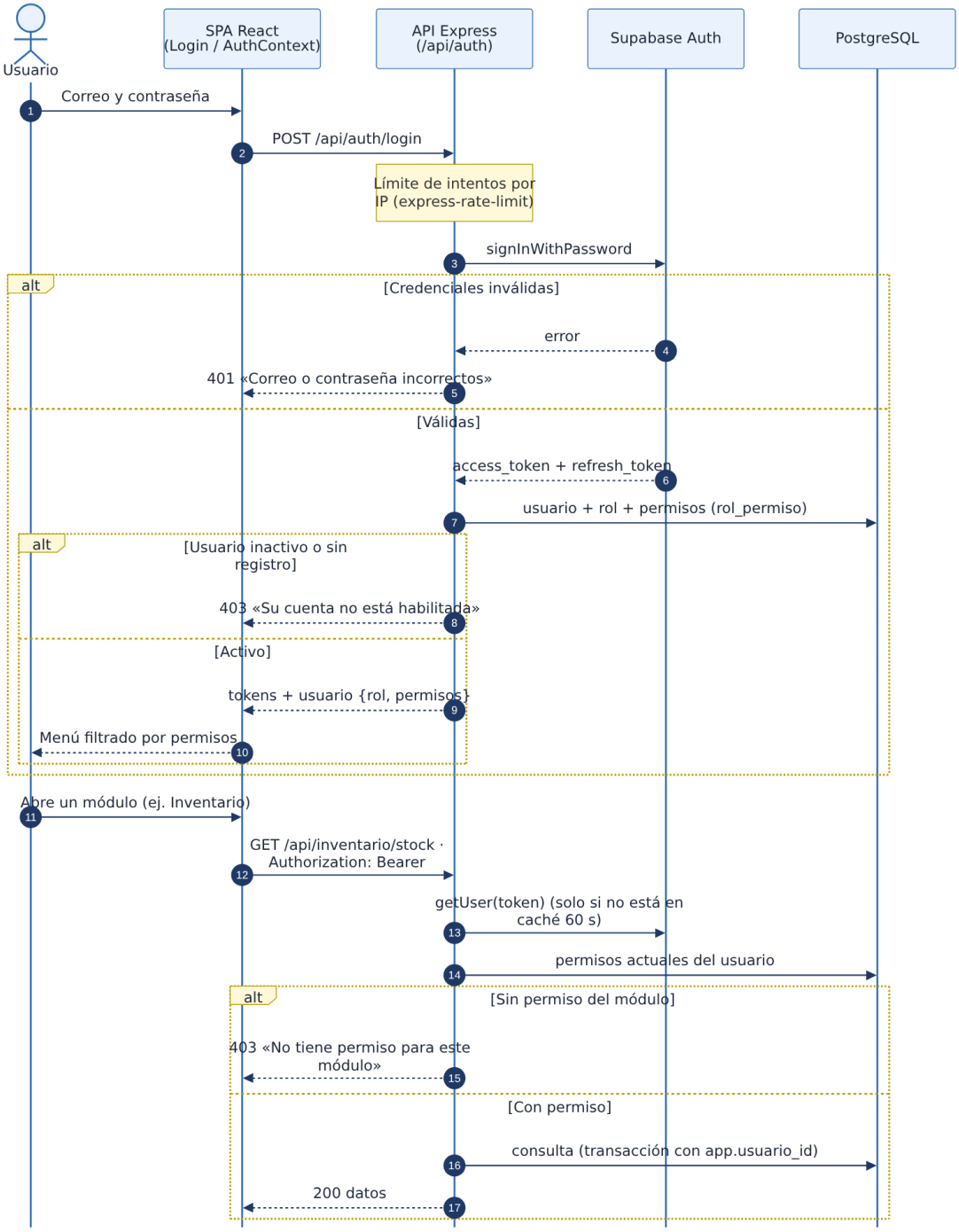


Figura 11. Secuencia: inicio de sesión y autorización por módulo.

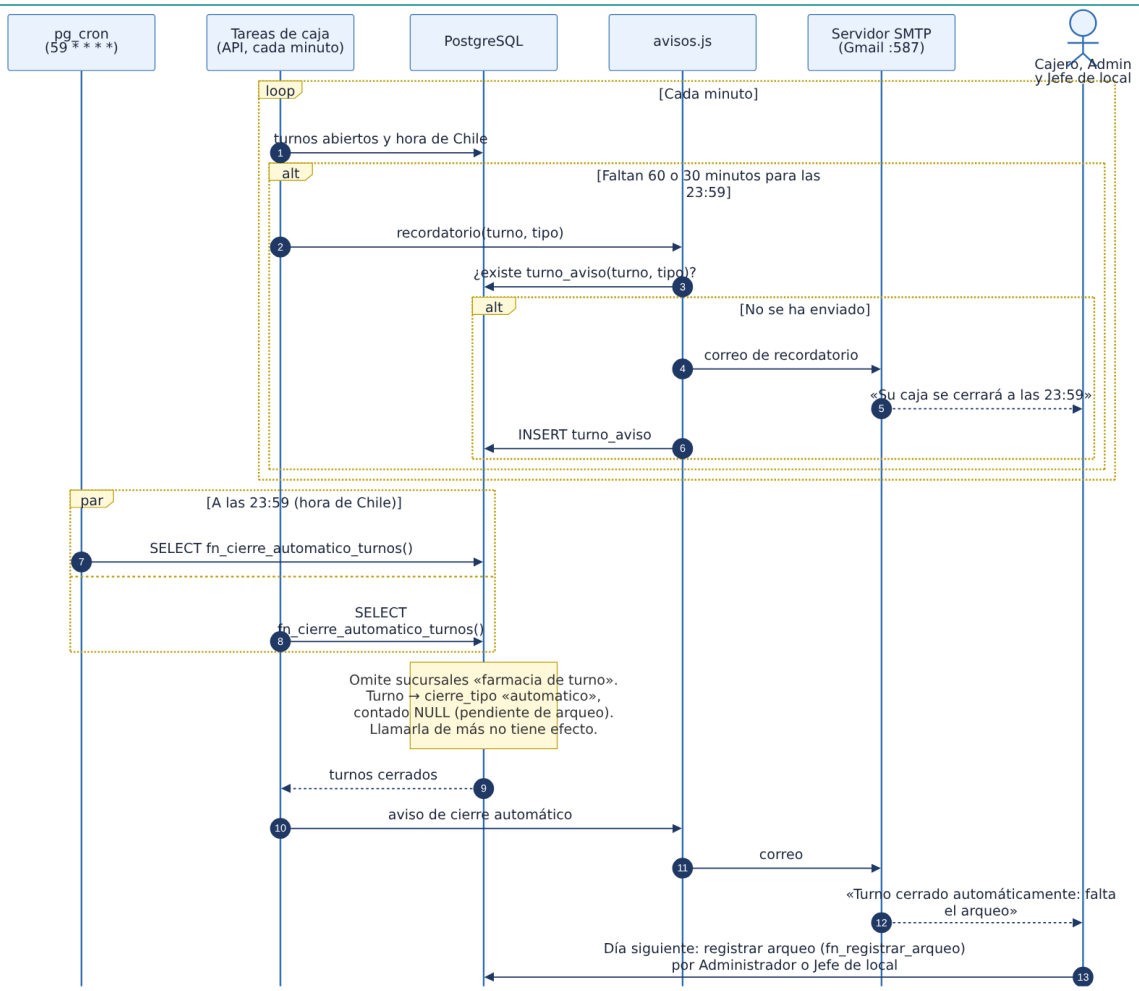


Figura 12. Secuencia: cierre automático y avisos de caja.

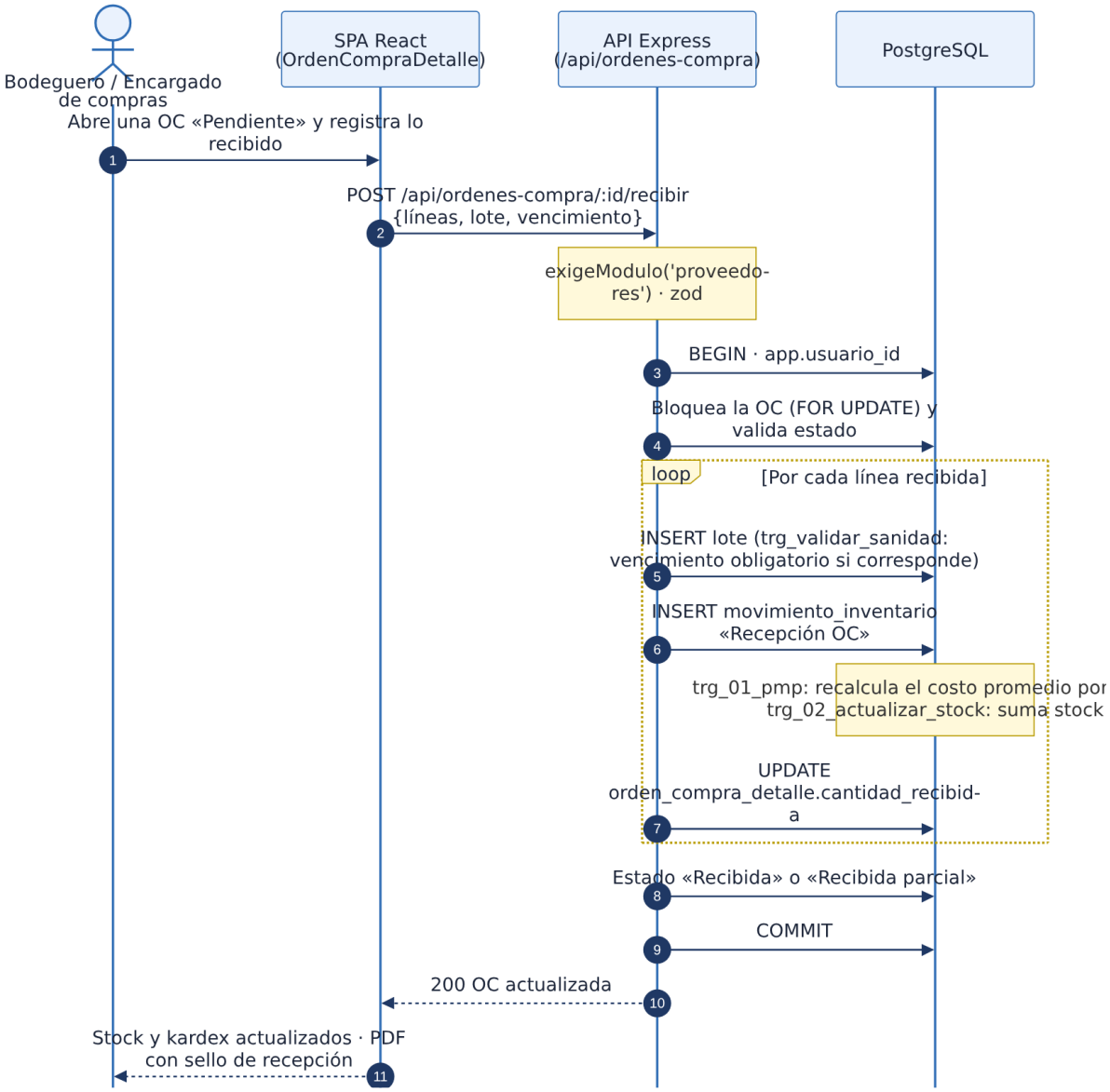


Figura 13. Secuencia: recepción de una orden de compra.

5.4 Diagramas de estados

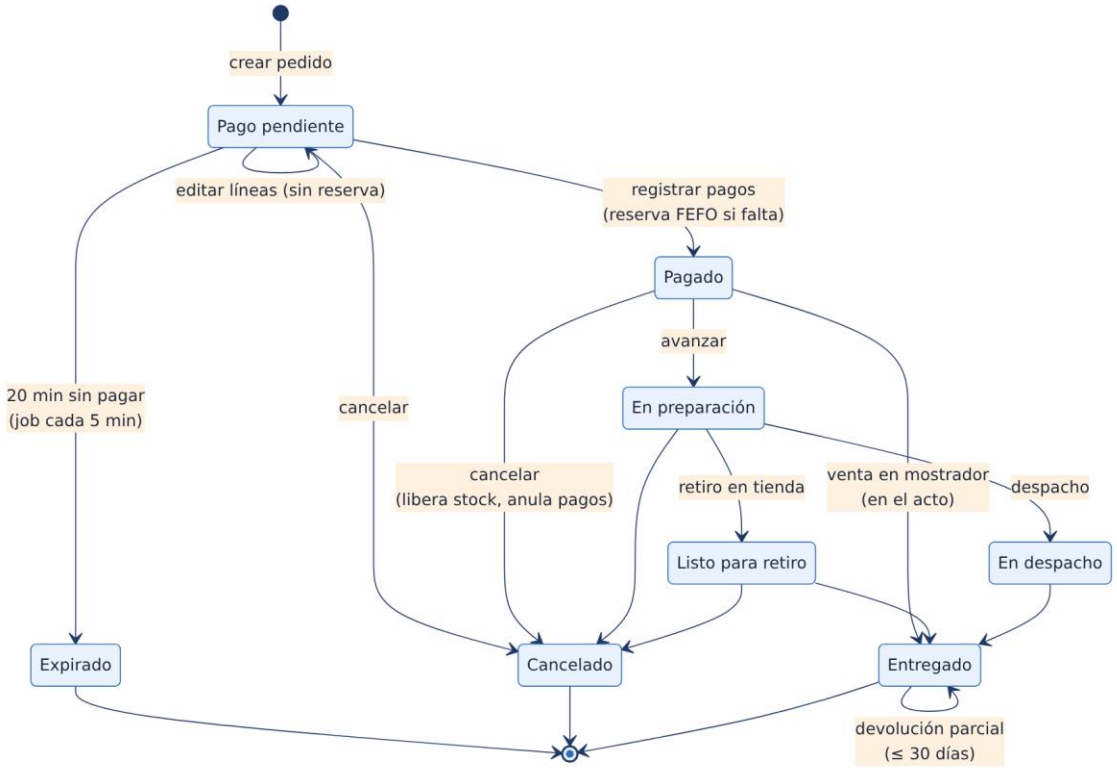


Figura 14. Estados del pedido.

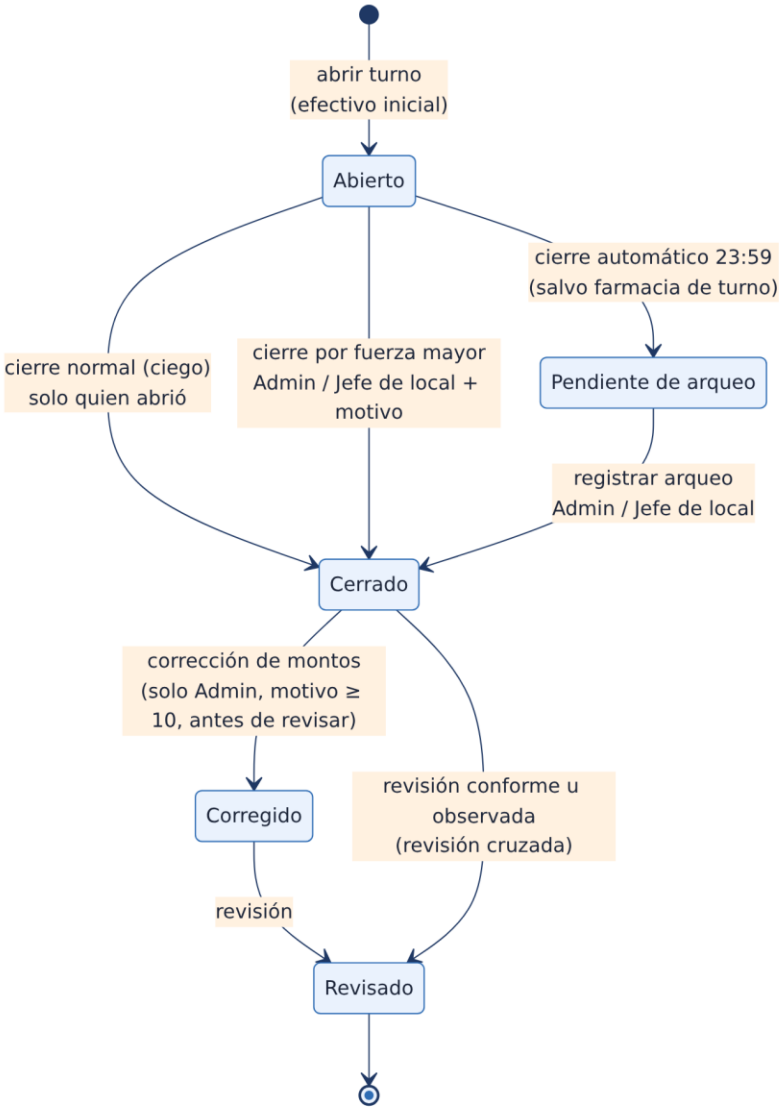


Figura 15. Estados del turno de caja.

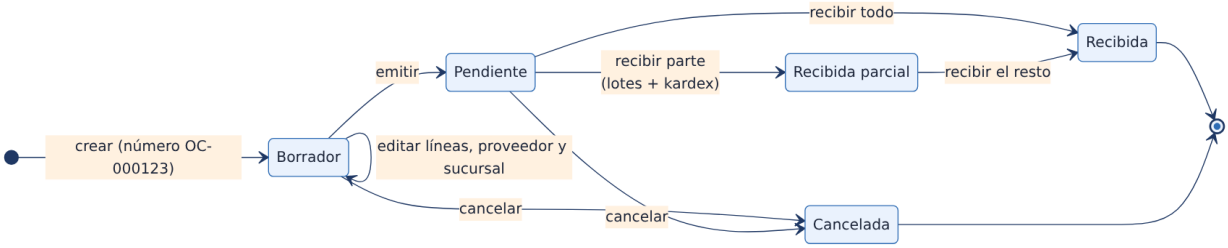


Figura 16. Estados de la orden de compra.

Los diagramas de actividad de los procesos de negocio están en el documento de flujos (DOC-10).

6. API REST

Prefijo `/api`. Todas las rutas, salvo `auth` y `health`, exigen `Authorization: Bearer <token>` y el permiso del módulo; las lecturas aceptan además los permisos de lectura cruzada declarados por el módulo.

Módulo - permiso	Rutas
auth (público)	POST /auth/login - POST /auth/refresh - GET /auth/me
usuarios - usuarios	/usuarios (GET, GET :id, POST, PUT :id) - /roles (GET, PUT :id/permisos)
proveedores - proveedores	GET - GET :id - POST - PUT :id
productos - inventario	GET - GET /proveedor/:id - GET :id - POST - PUT :id - PATCH :id/activo - PUT :id/proveedores
ordenes-compra - proveedores	GET - GET :id - POST - PUT :id - POST :id/emitir - /cancelar - /recibir - GET :id/pdf
inventario - inventario	/stock (GET, PUT /minimo) - /lotes (GET, POST :id/baja-vencido, POST /bajas-vencidos) - /kardex (GET) - /ajustes (GET, POST)
pedidos - pedidos	GET /catalogo - GET - GET :id - POST - PUT :id - POST /expirar-pendientes - POST :id/reservar - /pagos - /estado - /cancelar - /enviar - GET :id/pdf
pos - pos	GET /turno - POST /turnos - POST /turnos/:id/cerrar - GET /turno/ventas - GET /productos - POST /cotizar - POST /ventas - GET /ventas/:id/pdf - POST /ventas/:id/enviar - GET/POST /cajas - PATCH /cajas/:id - GET /sucursales - PATCH /sucursales/:id
devoluciones - devoluciones	GET - GET /pedidos-devolvibles - GET /pedido/:id - POST /cotizar - POST - GET :id - POST :id/enviar - GET :id/pdf
cuadratura - cuadratura	GET /turnos - /cajeros - /resumen - /turnos/:id - POST /turnos/:id/revision - /correcciones - /observaciones - /cierre-forzado - /arqueo - GET /turnos/:id/pdf - /excel - /exportar
dashboard - dashboard	GET /resumen - /informes - /informes/:id - /informes/:id/exportar
comunes	GET /catalogos (sesión) - GET /health - GET /health/db

7. Requisitos no funcionales

ID	Atributo	Requisito y cómo se cumple	Verificación
RNF-01	Seguridad	Autenticación con Supabase Auth; autorización por módulo leída en cada solicitud; RLS en todas las tablas con un único rol app_backend; HTTPS en producción; secretos solo en .env (chmod 600) y secretos de GitHub; límite de intentos de login; cabeceras de seguridad en Nginx.	Pruebas de permisos (403), ZAP baseline, job seguridad de la CI.
RNF-02	Integridad y auditoría	Reglas de negocio en la base (funciones SECURITY DEFINER y triggers); kardex, devoluciones, revisiones, correcciones y observaciones inmutables; cada acción registra el usuario (app.usuario_id).	Pruebas unitarias y SQL de verificación.
RNF-03	Rendimiento	Respuestas p95 < 800 ms con 20 usuarios concurrentes en la VM E2.1.Micro; consultas con índices y paginación; PDF/Excel generados en el servidor y enviados como base64.	Prueba de carga k6.
RNF-04	Disponibilidad	Contenedores con restart: unless-stopped y healthcheck; balanceador con health check /healthz; workflow que evita la pausa de Supabase y la recuperación de VMs ociosas.	Monitoreo del health check y de los workflows.
RNF-05	Escalabilidad	API sin estado (tokens), lista para más instancias detrás del balanceador; base gestionada en Supabase con pooler; módulos independientes.	Revisión de arquitectura.
RNF-06	Portabilidad	Todo corre en contenedores Docker (desarrollo y producción); imágenes linux/amd64 publicadas en GHCR; configuración por variables de entorno.	docker compose up en un PC limpio.
RNF-07	Mantenibilidad	Arquitectura por módulos en backend y frontend; migraciones numeradas e idempotentes; CI con pruebas en cada cambio; código y comentarios en español.	CI en verde.
RNF-08	Usabilidad	Interfaz responsive con Bootstrap; montos en palabras para evitar errores de digitación; mensajes de error claros (409 con explicación).	Pruebas con usuarios de MediFarm.
RNF-09	Cumplimiento legal	Redondeo del efectivo según la Ley 20.956; control de lotes y vencimientos.	Pruebas unitarias de redondeo.
RNF-10	Costo	Operación a costo cero con recursos Always Free de OCI, plan gratuito de Supabase, GitHub, DuckDNS y Let's Encrypt.	Presupuesto de USD 1 con alerta.

8. Docker y ejecución

8.1 Dockerfiles

Imagen	Base	Detalle
farmaclick-api (backend/Dockerfile)	node:22-alpine	npm ci --omit=dev; copia src y scripts/crear-admin.js; usuario node (sin privilegios); EXPOSE 3000; healthcheck GET /api/health; CMD node src/server.js
farmaclick-web (frontend/Dockerfile)	node:22-alpine (build) > nginx:1.27-alpine	Etapa 1 compila la SPA (npm run build); etapa 2 copia dist/ y deploy/nginx.conf; EXPOSE 80; healthcheck /healthz

8.2 docker-compose

Archivo	Servicios	Uso
docker-compose.yml (raíz)	db (postgres:16-alpine :54329), api (node:22 :3000, AUTH_MODE=local), web (Vite :5173)	Desarrollo local de los tres integrantes; la base se crea con instalacion_completa.sql, desarrollo.sql y datos-prueba.sql
deploy/docker-compose.yml	api (expose 3000, env_file .env) y web (80:80), imágenes de GHCR	VM dev y prod; lo copia la CI en cada despliegue

8.3 Variables de entorno

Variable	Uso	Ejemplo / valor
DATABASE_URL	Conexión por Transaction pooler con app_backend	postgresql://app_backend.<ref>:<clave>@aws-1-sa-east-1.pooler.supabase.com:6543/postgres
DATABASE_SSL	TLS hacia la base	true (local: false)
SUPABASE_URL / _ANON_KEY / _SERVICE_ROLE_KEY	Supabase Auth	del proyecto (secreto)
PORT / NODE_ENV	Puerto y modo	3000 / production
TRUST_PROXY	Saltos de proxy para el rate limit	1 en dev, 2 en prod (balanceador + Nginx)
CORS_ORIGIN	Origen permitido	https://farmaclick-medifarm.duckdns.org
EMPRESA_NOMBRE / _RUT / _DIRECCION	Encabezado de los PDF	FarmaClick
DEVOLUCION_MAX_DIAS	Plazo de devoluciones	30
SMTP_HOST / PORT / SECURE / USER / PASS / FROM	Correo saliente	smtp.gmail.com / 587 / false / ... (secreto)
TAREAS_CAJA	Desactivar tareas de caja	off (opcional)
GHCR_OWNER / IMAGE_TAG	Imágenes a usar	alefuentes982 / dev o prod (las inyecta la CI)
AUTH_MODE	Login de desarrollo	local (solo en docker compose local; prohibido con NODE_ENV=production)

8.4 Cómo levantar el sistema

```
git clone https://github.com/Alefuentes982/FarmaClick.git && cd FarmaClick
git checkout develop
docker compose up              # web http://localhost:5173 - API http://localhost:3000/api/health
docker compose exec api npm test
docker compose down -v         # base desde cero
```

Usuarios de desarrollo: `admin@farmaclick.local` / `Admin12345` y `vendedor@farmaclick.local` / `Vendedor12345`. El detalle del despliegue en la nube está en el manual técnico (DOC-09).

9. Decisiones de arquitectura

#	Decisión	Motivo	Consecuencia
AD-01	Reglas críticas en la base de datos	Ningún cliente puede saltarse precio, stock o cierre de caja	Más lógica en SQL; se prueba con el job esquema de la CI
AD-02	Registro de módulos con permiso propio	Agregar módulos sin tocar los existentes	Cada módulo declara nav, rutas, permiso y catálogos
AD-03	Permisos leídos en cada solicitud	Quitar un permiso rige de inmediato	Una consulta extra por solicitud (barata)
AD-04	Transaction pooler IPv4	La conexión directa (IPv6) falla desde OCI	Sin sentencias preparadas con nombre
AD-05	Archivos como base64 dentro de JSON	Los gestores de descarga (IDM) vaciaban las descargas	Archivos pequeños; el frontend los convierte a Blob
AD-06	Imágenes linux/amd64	Las VM Always Free usadas son E2.1.Micro x86_64	Build con QEMU/buildx en la CI
AD-07	El despliegue no toca la base	Evitar migraciones accidentales en producción	Migraciones manuales: dev primero, prod antes del merge
AD-08	Trabajo directo en develop + merge fast-forward a main	Equipo pequeño; la CI valida cada push	Plantilla de PR y CODEOWNERS disponibles si se adoptan ramas

Anexo A. Diccionario de datos

caja - Cajas físicas de cada sucursal.

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
sucursal_id	uuid	No	FKUQ	sucursal
numero	integer	No	UQ	-
activa	boolean	No		-

canal_venta - Presencial (exige turno) u Online.

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
nombre	varchar(50)	No	UQ	-
requiere_turno	boolean	No		-

categoria - Categorías con márgenes pesimista, probable y optimista (PERT).

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
nombre	varchar(100)	No	UQ	-
activa	boolean	No		-
margen_pesimista	numeric(4,2)	No		-
margen_probable	numeric(4,2)	No		-
margen_optimista	numeric(4,2)	No		-

cliente - Datos del cliente final (para la tienda en línea).

Columna	Tipo	Nulo	Clave	Referencia
usuario_id	uuid	No	PK FK	usuario
rut	varchar(12)	Sí	UQ	-
telefono	varchar(30)	Sí		-
direccion_cliente	varchar(200)	Sí		-
comuna_id	uuid	Sí	FK	comuna

comuna - Comunas de Chile.

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
nombre	varchar(100)	No	UQ	-
region_id	uuid	No	FKUQ	region

condicion_venta - Condición de venta (venta directa, receta, etc.) y si permite venta en línea.

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
nombre	varchar(50)	No	UQ	-
permite_venta_online	boolean	No		-
activa	boolean	No		-

detalle_pedido - Líneas del pedido con precio y descuento fijados por la base.

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
nombre_producto	varchar(200)	No		-
cantidad	integer	No		-
precio_unitario	numeric(12,2)	No		-
descuento_monto	numeric(12,2)	No		-
pedido_id	uuid	No	FK	pedido
producto_id	uuid	No	FK	producto
regla_descuento_id	uuid	Sí	FK	regla_descuento

devolucion - Devolución inmutable de un pedido entregado con total y redondeo.

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
numero	bigint	No		-
fecha	timestamptz	No		-
pedido_id	uuid	No	FK	pedido
usuario_id	uuid	No	FK	usuario
turno_caja_id	uuid	Sí	FK	turno_caja
motivo	varchar(300)	No		-
total_reembolso	numeric(12,2)	No		-
redondeo	numeric(12,2)	No		-

devolucion_detalle - Líneas devueltas, reembolso y si reingresa stock.

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
devolucion_id	uuid	No	FK	devolucion
detalle_pedido_id	uuid	No	FK	detalle_pedido
cantidad	integer	No		-
monto_reembolso	numeric(12,2)	No		-
reingresa_stock	boolean	No		-

devolucion_pago - Reembolso por medio de pago.

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
devolucion_id	uuid	No	FK	devolucion
medio_pago_id	uuid	No	FK	medio_pago
monto	numeric(12,2)	No		-
referencia	varchar(50)	Sí		-

estado_orden_compra - Borrador, Pendiente, Recibida parcial, Recibida, Cancelada.

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
nombre	varchar(50)	No	UQ	-

estado_pedido - Pago pendiente, Pagado, En preparación, Listo para retiro, En despacho, Entregado, Cancelado, Expirado.

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
nombre	varchar(50)	No	UQ	-

estado_transaccion - Estados de un pago (Aprobado, Anulado, Abortado...).

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
nombre	varchar(50)	No	UQ	-

lote - Lotes por producto y sucursal con vencimiento y cantidad actual.

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
numero_lote	varchar(50)	Sí	UQ	-
fecha_vencimiento	date	Sí		-
fecha_ingreso	date	No		-
cantidad_recibida	integer	No		-
cantidad_actual	integer	No		-
producto_id	uuid	No	FKUQ	producto
sucursal_id	uuid	No	FKUQ	sucursal
orden_compra_detalle_id	uuid	Sí	FK	orden_compra_detalle

medio_pago - Efectivo, Débito, Crédito, Transferencia, Webpay Plus.

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
nombre	varchar(50)	No	UQ	-
activo	boolean	No		-

modulo - Módulos con clave de permiso (dashboard, pos, inventario, proveedores, pedidos, cuadratura, usuarios, devoluciones, supervision_caja).

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
nombre	varchar(100)	No		-
clave	varchar(50)	No	UQ	-

motivo_movimiento - Motivos (recepción, venta, ajuste, merma, vencido, devolución...).

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
nombre	varchar(100)	No	UQ	-
tipo_movimiento_id	uuid	No	FK	tipo_movimiento
activo	boolean	No		-

movimiento_inventario - Kardex inmutable: cada entrada y salida con motivo, lote y usuario.

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
producto_id	uuid	No	FK	producto
sucursal_id	uuid	No	FK	sucursal
cantidad	integer	No		-

Columna	Tipo	Nulo	Clave	Referencia
fecha	timestamptz	No		-
motivo_movimiento_id	uuid	No	FK	motivo_movimiento
lote_id	uuid	No	FK	lote
usuario_id	uuid	No	FK	usuario
pedido_id	uuid	Sí	FK	pedido
orden_compra_id	uuid	Sí	FK	orden_compra
observacion	varchar(300)	Sí		-

orden_compra - Orden de compra con número correlativo, estado y totales.

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
fecha	timestamptz	No		-
fecha_esperada	date	Sí		-
estado_orden_compra_id	uuid	No	FK	estado_orden_compra
subtotal_neto	numeric(12,2)	No		-
iva	numeric(12,2)	No		-
total	numeric(12,2)	No		-
proveedor_id	uuid	No	FK	proveedor
usuario_id	uuid	No	FK	usuario
sucursal_id	uuid	No	FK	sucursal
numero	bigint	No		-

orden_compra_detalle - Líneas de la OC: solicitado, recibido y costo.

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
cantidad_solicitada	integer	No		-
cantidad_recibida	integer	No		-
costo_unitario	numeric(12,2)	No		-
orden_compra_id	uuid	No	FK	orden_compra
producto_id	uuid	No	FK	producto

pasarela - Pasarelas de pago (Webpay Plus, tienda en línea).

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
nombre	varchar(50)	No	UQ	-
activa	boolean	No		-

pedido - Venta (presencial u online) con número, estado, totales, redondeo y turno de caja.

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
fecha	timestamptz	No		-
canal_venta_id	uuid	No	FK	canal_venta
estado_pedido_id	uuid	No	FK	estado_pedido

Columna	Tipo	Nulo	Clave	Referencia
subtotal_bruto	numeric(12,2)	No		-
total	numeric(12,2)	No		-
subtotal_neto	numeric(12,2)	No		-
iva	numeric(12,2)	No		-
nombre_cliente	varchar(150)	No		-
direccion_pedido	varchar(200)	Sí		-
telefono	varchar(30)	Sí		-
comuna_id	uuid	Sí	FK	comuna
cliente_id	uuid	Sí	FK	usuario
vendedor_id	uuid	Sí	FK	usuario
sucursal_id	uuid	Sí	FK	sucursal
turno_caja_id	uuid	Sí	FK	turno_caja
numero	bigint	No		-
redondeo	numeric(12,2)	No		-

producto - Catálogo: SKU, costo, margen, precio calculado, control de lote y vencimiento.

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
sku	varchar(50)	No	UQ	-
nombre	varchar(200)	No		-
costo	numeric(12,2)	No		-
margen_utilidad	numeric(4,2)	No		-
precio	numeric(12,2)	Sí		-
activo	boolean	No		-
stock_total_web	integer	No		-
requiere_vencimiento	boolean	No		-
requiere_lote	boolean	No		-
categoria_id	uuid	No	FK	categoria
condicion_venta_id	uuid	No	FK	condicion_venta
creado_en	timestamptz	No		-

producto_proveedor - Proveedores de cada producto, costo referencial y proveedor principal.

Columna	Tipo	Nulo	Clave	Referencia
producto_id	uuid	No	PK FK	producto
proveedor_id	uuid	No	PK FK	proveedor
costo_referencial	numeric(12,2)	No		-
es_principal	boolean	No		-

proveedor - Proveedores con RUT validado.

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
nombre	varchar(150)	No		-
rut	varchar(12)	No	UQ	-

Columna	Tipo	Nulo	Clave	Referencia
contacto	varchar(150)	Sí		-
activo	boolean	No		-

region - Regiones de Chile.

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
nombre	varchar(100)	No	UQ	-

regla_descuento - Descuentos por cantidad, opcionalmente por categoría, con vigencia.

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
nombre	varchar(50)	No		-
categoria_id	uuid	Sí	FK	categoria
cantidad_minima	integer	No		-
cantidad_maxima	integer	Sí		-
porcentaje_descuento	numeric(5,2)	No		-
fecha_inicio	timestamptz	Sí		-
fecha_fin	timestamptz	Sí		-
activa	boolean	No		-

rol - Roles: Administrador, Vendedor, Bodega, Cliente, Sistema y Jefe de local.

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
nombre	varchar(50)	No	UQ	-

rol_permiso - Matriz de permisos rol x módulo.

Columna	Tipo	Nulo	Clave	Referencia
rol_id	uuid	No	PK FK	rol
modulo_id	uuid	No	PK FK	modulo
permitido	boolean	No		-

stock_minimo_sucursal - Stock mínimo por producto y sucursal.

Columna	Tipo	Nulo	Clave	Referencia
producto_id	uuid	No	PK FK	producto
sucursal_id	uuid	No	PK FK	sucursal
stock_minimo	integer	No		-

sucursal - Locales de la farmacia; farmacia_de_turno = atiende 24 h.

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
nombre	varchar(150)	No		-
direccion	varchar(200)	Sí		-

Columna	Tipo	Nulo	Clave	Referencia
telefono	varchar(30)	Sí		-
activa	boolean	No		-
habilita_venta_web	boolean	No		-
comuna_id	uuid	No	FK	comuna
farmacia_de_turno	boolean	No		-

tipo_movimiento - Entrada o salida (signo).

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
nombre	varchar(50)	No	UQ	-
signo	smallint	No		-

transaccion_pago - Pagos del pedido por medio y pasarela, con estado.

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
pedido_id	uuid	No	FK	pedido
pasarela_id	uuid	Sí	FK	pasarela
medio_pago_id	uuid	No	FK	medio_pago
buy_order	varchar(26)	Sí	UQ	-
token_inicio	varchar(100)	Sí		-
codigo_autorizacion	varchar(50)	Sí		-
estado_transaccion_id	uuid	No	FK	estado_transaccion
monto	numeric(12,2)	No		-
fecha_transaccion	timestamptz	No		-

turno_aviso - Avisos de cierre enviados (evita repetirlos).

Columna	Tipo	Nulo	Clave	Referencia
turno_caja_id	uuid	No	PK FK	turno_caja
tipo	varchar(30)	No	PK	-
enviado_en	timestamptz	No		-
destinatarios	integer	No		-

turno_caja - Turno de caja: apertura, cierre, esperado, contado, tipo de cierre y arqueo.

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
usuario_id	uuid	No	FK	usuario
caja_id	uuid	No	FK	caja
fecha_apertura	timestamptz	No		-
fecha_cierre	timestamptz	Sí		-
monto_apertura_efectivo	numeric(12,2)	No		-
monto_cierre_esperado_efectivo	numeric(12,2)	No		-
monto_cierre_real_efectivo	numeric(12,2)	Sí		-
cierre_tipo	varchar(12)	Sí		-

Columna	Tipo	Nulo	Clave	Referencia
cerrado_por	uuid	Sí	FK	usuario
cierre_motivo	varchar(500)	Sí		-
arqueo_por	uuid	Sí	FK	usuario
arqueo_fecha	timestamptz	Sí		-

turno_correccion - Correcciones de montos con motivo (inmutable).

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
turno_caja_id	uuid	No	FK	turno_caja
usuario_id	uuid	No	FK	usuario
fecha	timestamptz	No		-
campo	varchar(10)	No		-
valor_anterior	numeric(12,2)	No		-
valor_nuevo	numeric(12,2)	No		-
esperado_anterior	numeric(12,2)	Sí		-
esperado_nuevo	numeric(12,2)	Sí		-
motivo	varchar(500)	No		-

turno_observacion - Observaciones del turno (inmutables).

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
turno_caja_id	uuid	No	FK	turno_caja
usuario_id	uuid	No	FK	usuario
fecha	timestamptz	No		-
texto	varchar(1000)	No		-

turno_revision - Revisión conforme u observada (inmutable).

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
turno_caja_id	uuid	No	FKUQ	turno_caja
usuario_id	uuid	No	FK	usuario
fecha	timestamptz	No		-
conforme	boolean	No		-
observacion	varchar(500)	Sí		-

usuario - Personal del sistema (mismo id que la cuenta de Supabase Auth), con rol y sucursal.

Columna	Tipo	Nulo	Clave	Referencia
id	uuid	No	PK	-
nombre	varchar(150)	No		-
correo	varchar(150)	No	UQ	-
rol_id	uuid	No	FK	rol
sucursal_id	uuid	Sí	FK	sucursal
activo	boolean	No		-

Columna	Tipo	Nulo	Clave	Referencia
creado_en	timestamptz	No		-